EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh

Mini Project Report
submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Computer Engineering

by

Twissha Shah (UID 2024300231)
Miti Shah (UID 2024300230)
Tanvi Sheth (UID 2024300239)

Group 20 • T.E. Semester V • Mini Project – I

under the guidance of

Prof. Anuj Tawari



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058

Academic Year 2026–27

Certificate

This is to certify that the Mini Project entitled "EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh" by Twissha Shah, Miti Shah and Tanvi Sheth is approved for the partial fulfillment of Mini Project – I for the Third Year Engineering Project under the guidance of Prof. Anuj Tawari for the award of the Degree of Bachelor of Technology in Computer Engineering from the University of Mumbai.

Certified by

Prof. Anuj Tawari
Project Guide

Dr. Sudhir Dhage
Head of Department

Dr. B. N. Chaudhari
Principal



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri(W), Mumbai-400058
Academic Year 2026–27

Approval Certificate

This is to certify that the Mini Project entitled "EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh" by Twissha Shah, Miti Shah and Tanvi Sheth is approved for the partial fulfillment of Mini Project – I for the Third Year Mini Project towards obtaining the Degree of Bachelor of Technology in Computer Engineering.

Project Guide

(signature)

Name:

Date:

External Examiner

(signature)

Name:

Date:

Head of Department

Principal

Seal of the Institute

Declaration

We wish to state that the work embodied in this work titled "EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh" forms our own contribution to the work carried out under the guidance of Prof. Anuj Tawari at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Twissha Shah **Miti Shah** **Tanvi Sheth**
UID 2024300231 **UID 2024300230** **UID 2024300239**

Acknowledgement

We would like to express our sincere gratitude to our project guide, Prof. Anuj Tawari, for the direction, the regular reviews and the candid feedback that shaped this project from a problem statement into a working, measured system. His insistence that every claim be backed by evidence is reflected throughout this report.

We thank the Head of the Department of Computer Engineering and the Principal of the Sardar Patel Institute of Technology for providing the environment, the laboratory facilities and the opportunity to carry out this mini project. We are grateful to the faculty who coordinated the evaluation phases and the report guidelines, and to the examiners whose comments at the earlier phases improved the design.

Finally, we thank our classmates, who tested the demonstration and gave honest feedback on the interface, and our families for their support throughout the semester.

Twissha Shah
Miti Shah
Tanvi Sheth

Abstract

Problem. When an earthquake, fire or building collapse destroys communication infrastructure, every mainstream messaging application fails with it, even when the devices of the trapped people can still reach each other. Connectivity after a disaster survives only in scattered pockets and normally helps only the person who holds it.

Motivation and objectives. Rescue is time-critical, so the aim is a system that makes one person's connectivity serve everyone and lets an SOS overtake routine traffic at every hop: self-organising, link-quality-weighted routing, reliable delivery with a delay-tolerant fallback, authenticated and encrypted messages, and measured performance.

Proposed solution. EcoMesh makes every participating device an endpoint and a relay. A message hops from device to device until it reaches a gateway with an outside link, a rescue device, or a store-and-forward queue that holds it until a path appears. No internet, tower or server is needed.

Technologies and engineering concepts. Python with `asyncio` (one event-loop thread per node, no locks); a custom 52-byte binary protocol; link-state gossip; Dijkstra routing over the cost $1 + 5(1 - q) + 3f$; a per-hop priority queue; ACK with retry and route recomputation; AES-256-GCM and Ed25519; per-node SQLite logs; FastAPI and WebSocket; a React web application; and a transport interface so one engine runs on an in-memory simulator, on loopback sockets and on separate laptops.

Key implementation features. Neighbour discovery with measured link quality; network-wide link-state flooding with sequence numbers; duplicate suppression; bounded store-and-forward that never evicts an SOS for routine traffic; signatures that survive multi-hop relaying; a survivor screen with a hold-to-send SOS button and a rescue console.

Testing methodology. 312 automated tests (unit, integration, boundary, fuzz, replay, concurrency and a three-node mesh over real TCP), a requirement-to-test table for 68 requirements, seven simulator scenarios with seeded repeated runs, loss and scale sweeps up to 65 nodes, and measurements on five real processes over real sockets.

Results. On a 40-node mesh with 20 % per-link loss and injected failures, 92.6 % of messages were delivered (50 runs). Weighted routing delivered 99.2 % against 57.4 % for hop-count routing on identical seeds. Under load an SOS had a 4.4× lower median latency than routine traffic. On five live processes an SOS arrived in a median of 47 ms, the sender re-routed around a failed relay in a median of 4.0 s, and every held message was delivered after a gateway outage. Convergence and recovery met their targets on average but not in every run, and delivery fell below 90 % above about 43 nodes at 20 % loss.

Practical applicability and limitations. The system is directly usable as an institutional safety network on laptops. The radio link is emulated by a neighbour-restricted LAN, so it does not yet run phone to phone; a Bluetooth or Wi-Fi Direct transport, adaptive timers, a timed usability study and per-pair key agreement are the main future work.

Contents

Abstract **5**

List of Abbreviations and Symbols **11**

1 Introduction and Problem Definition **12**

1.1 Project Background . . . 12
1.2 Problem Statement . . . 12
1.3 Motivation . . . 12
1.4 Objectives . . . 12
1.5 Scope of the Project . . . 12
1.6 Proposed Solution . . . 13
1.7 Novelty and Originality . . . 13
1.8 Feasibility Analysis . . . 13
1.9 Social Relevance and Practical Applications . . . 13
1.10 Success Criteria and Key Performance Indicators . . . 13

2 Literature Review and Study of Existing Systems **15**

2.1 Study of Existing Systems . . . 15
2.2 Review of Research Papers and Technical Resources . . . 15
2.3 Comparative Analysis . . . 16
2.4 Limitations of Existing Systems . . . 16
2.5 Identified Research or Engineering Gap . . . 16
2.6 Proposed Improvements . . . 16

3 Software Requirements Specification **18**

3.1 Purpose and Scope of the SRS . . . 18
3.2 Intended Users and Stakeholders . . . 18
3.3 Overall Product Description . . . 18
3.4 Functional Requirements . . . 18
3.5 Non-Functional Requirements . . . 21
3.6 Hardware and Software Requirements . . . 21
3.7 Constraints and Assumptions . . . 21
3.8 Use Cases and User Stories . . . 22

4 System Architecture and Design **23**

4.1 Overall System Architecture . . . 23
4.2 Component and Module Description . . . 23
4.3 Architecture Selection and Justification . . . 23
4.4 Data Flow Diagram (DFD) . . . 25
4.5 UML Diagrams . . . 25
4.6 Database Design . . . 28
4.7 Security and Error Handling Design . . . 29

5 Algorithm Design and Data Structures **31**

5.1 Identification of Critical Operations . . . 31
5.2 Algorithm Design . . . 31
5.3 Flowcharts . . . 32
5.4 Data Structure Selection . . . 32
5.5 Algorithm Selection and Justification . . . 32
5.6 Time Complexity Analysis . . . 34
5.7 Space Complexity Analysis . . . 34

5.8 Optimization Opportunities . . . 35
6 Subject-Wise Engineering Concepts and APNI Leap Alignment 36
6.1 Software Engineering . . . 36
6.2 Computer Networks . . . 36
6.3 Database Management Systems . . . 36
6.4 Operating Systems . . . 36
6.5 Data Structures . . . 37
6.6 Design and Analysis of Algorithms . . . 37
6.7 Cloud Computing . . . 37
6.8 Mapping of Subjects to Project Implementation . . . 37
7 Implementation and Module-Wise Coding 38
7.1 Development Environment . . . 38
7.2 Overall Implementation Strategy . . . 38
7.3 Module-Wise Implementation . . . 38
7.4 Front-End Implementation . . . 39
7.5 Back-End Implementation . . . 39
7.6 Database Implementation . . . 39
7.7 Communication and Concurrency Implementation . . . 39
7.8 Git Repository Setup . . . 40
7.9 Integration of Modules . . . 40
7.10 Implementation Challenges and Resolutions . . . 40
8 Scalability, Performance and Resource Management 41
8.1 Scalability Requirements . . . 41
8.2 Workload and Load Model . . . 41
8.3 Resource Utilization . . . 41
8.4 Concurrent Processing . . . 41
8.5 Scheduling and Synchronization Analysis . . . 41
8.6 Performance Measurement . . . 41
8.7 Performance Results . . . 42
8.8 Bottleneck Identification . . . 44
8.9 Optimization and Improvement . . . 44
8.10 Cloud Deployment and Resource Utilization . . . 44
8.11 Scalability Limitations . . . 44
9 Testing and Validation 45
9.1 Testing Strategy . . . 45
9.2 Unit Testing . . . 45
9.3 Integration Testing . . . 45
9.4 Functional Testing . . . 45
9.5 Non-Functional Testing . . . 45
9.6 Boundary and Invalid-Input Testing . . . 46
9.7 Concurrency and Stress Testing . . . 46
9.8 Test Case Specification . . . 46
9.9 Defect Identification and Resolution . . . 47
9.10 Validation Against KPIs . . . 48
9.11 Limitations and Untested Scenarios . . . 48
10 Results, Demonstration and Discussion 49
10.1 Implemented Features . . . 49

10.2 Screenshots and Demonstration Evidence 49
10.3 Functional Results 49
10.4 Performance Results 49
10.5 Comparison with Existing Systems 49
10.6 Achievement of Objectives 51
10.7 Discussion of Findings 51
10.8 Practical Applicability 51
11 Conclusion and Future Scope **53**
11.1 Conclusion 53
11.2 Achievement of Objectives 53
11.3 Major Engineering Contributions 53
11.4 Current Limitations 53
11.5 Future Enhancements 53
11.6 Lessons Learned 53
References **53**

List of Figures

1 Block diagram: one EcoMesh node and its three interchangeable transports . . . 23
2 Level-0 (context) data flow diagram . . . 25
3 Level-1 data flow diagram of one node . . . 25
4 Use case diagram . . . 26
5 Class diagram: `Node` as the composition root . . . 26
6 Sequence diagram: an SOS relayed hop by hop with the ACK returning along the reverse path . . . 27
7 Activity diagram: the packet receive path . . . 27
8 State machine of the message lifecycle (left) and deployment diagram of the live configuration (right) . . . 28
9 Entity–relationship diagram of the per-node database (PK primary key, IDX secondary index, FK logical foreign key) . . . 28
10 The cost function, and why hop count is a poor proxy for delivery . . . 32
11 Flowchart: originate, acknowledge, retry and retain a message . . . 33
12 Flowchart: next-hop selection, including `RESCUE__` . . . 33
13 (a) Delivery against per-link loss. (b) Weighted versus hop-count routing on identical seeds. (c) Delivery as the mesh grows. (d) Recovery time after a relay death, 50 trials. . . . 43
14 The survivor's screen. Left: SOS delivered over Phone 2 and the Gateway. Centre: the gateway has failed; the screen says "Searching for a path" and the message is "Saved, waiting for a path". Right: the gateway is back and the held message is delivered. . . . 50
15 The rescue console. Left: SOS cards with sender, text, age and route, counters, network map and device list. Right: with the gateway down the rescue device hears nobody ("1 of 5 devices reachable"); earlier messages remain visible. . . . 50
16 The engine web application. Left: rescue dashboard (delivery ratio, nodes online, live mesh, SOS queue). Centre: routing page (weighted versus hop-count route, cost function, a node's view of the graph). Right: the six-beat live demonstration page, which drives the real API. . . . 51

List of Tables

2 Key performance indicators . . . 14
3 Fifteen studies: technique, contribution and relevance to EcoMesh . . . 15
4 Existing offline-communication systems and routing protocols compared with EcoMesh (n/d = not documented in the sources consulted) . . . 17
5 Functional requirements by feature (identifiers as in the SRS). Check: Auto = verified by automated tests, Auto+UI = automated tests plus a manual check of the screen, Partial = only partly implemented, Inspect = code inspection, Manual = manual check only; * marks a mapped test that failed once in repeated runs. The first mapped test is named and (+n) counts the others. . . . 18
6 Modules, purpose (input → output) and dependencies . . . 24
7 The 52-byte packet header . . . 24
8 Sample rows of `messages` for one SOS from A to C in a real run: each node holds its own view of the same message id . . . 29
9 Threat model and countermeasures . . . 30
10 Data structures and why they were chosen . . . 34
11 Time complexity (V nodes, E links, n queued packets, q retained messages, D distinct destinations, $|N|$ neighbours) . . . 34
12 Subjects, concepts applied, modules and evidence . . . 37
13 Modules: code reference, logic, dependencies and tests . . . 38
14 Commits per team member, read from `git log` over all remote branches (49 commits) . . . 40
15 Reliability: delivery ratio as per-link loss increases (21-node grid, 8 seeded runs per point, ACK and retry enabled) . . . 42
16 Results of the seven simulator scenarios (weighted routing; delivery ratio shown as mean (min–max) over the runs) . . . 42
17 Latency by priority under load (16 nodes, one packet per 80 ms radio, 90 messages of which 15 % are SOS, all to the same destination; mean over 20 runs of the per-run statistic) . . . 43
18 Scalability, 10 to 65 simulated nodes (6 runs per size and loss rate) . . . 43
19 Live devices: 5 operating-system processes, real TCP sockets, real timers, on one laptop (loopback) . . . 44
20 Representative test cases (every requirement's tests are in Table 5) . . . 46
21 Significant defects, causes, corrective actions and retesting . . . 47
22 Validation of the key performance indicators of Table 2 against measured data . . . 48
23 Feature status . . . 49
24 Objectives, supporting evidence and status . . . 51

List of Abbreviations and Symbols

Term	Meaning
ACK	Acknowledgement packet returned by the destination along the reverse path
AES-256-GCM	Advanced Encryption Standard, 256-bit key, Galois/Counter Mode (authenticated encryption)
AODV	Ad hoc On-Demand Distance Vector routing (RFC 3561)
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface (used by FastAPI/uvicorn)
BLE	Bluetooth Low Energy
DAA	Design and Analysis of Algorithms
DBMS	Database Management System
DFD	Data Flow Diagram
DTN	Delay-Tolerant Networking (RFC 4838)
Ed25519	Edwards-curve Digital Signature Algorithm over Curve25519
ER	Entity–Relationship (diagram)
ESE	End Semester Evaluation
FR / NFR	Functional / Non-Functional Requirement
GATEWAY	Node role: has a link to the outside world; valid terminal for `RESCUE__`
HELLO	Periodic one-hop beacon used for neighbour discovery
KPI	Key Performance Indicator
LAN	Local Area Network
MANET	Mobile Ad hoc Network
OLSR	Optimized Link State Routing (RFC 3626)
OS	Operating System
PWA	Progressive Web App
RESCUE	Node role: rescue-team device; valid terminal for `RESCUE__`
`RESCUE__`	Symbolic destination meaning “the nearest GATEWAY or RESCUE node”
SF / S&F	Store-and-Forward
SOS	Highest message priority (value 0)
SRS	Software Requirements Specification
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
TTL	Time To Live (remaining hop allowance of a packet)
UML	Unified Modeling Language
WAL	Write-Ahead Logging (SQLite journal mode)
V, E	Number of vertices (nodes) and edges (links) of the mesh graph
q	Link quality in $[0, 1]$: fraction of the last 20 expected HELLOs received
f	Recent failures: ACK timeouts attributed to a link in the last 60 s
C_{max}, T_{hold}	Store-and-forward bounds: maximum count (500) and maximum age (1800 s)

1 Introduction and Problem Definition

1.1 Project Background

After an earthquake, fire or building collapse, cellular towers and internet links are unavailable or overwhelmed, yet connectivity is rarely zero: it survives in pockets (one person near a window with a bar of signal, a UPS-powered access point, a rescue team's own equipment) and normally helps only the person holding it. **EcoMesh** makes every participating device an endpoint *and* a relay, so a message hops from device to device until it reaches its destination, a *gateway* with an outside link, or a *rescue* device, with no internet and no server. The project comprises a complete engine (discovery, routing, reliability, security, storage), a simulator that measures it, a web interface for survivors and rescue coordinators, and a deployment in which every device is a separate process exchanging bytes only over TCP.

1.2 Problem Statement

WhatsApp, Signal, Telegram and SMS all need a message to reach a server or tower elsewhere; a Wi-Fi router with a cut uplink breaks every one of them although the devices on it can still reach each other. People trapped in a building therefore cannot signal their condition, share the one weak signal, coordinate, or be located efficiently. Existing offline systems flood or use hop-count routing, offer no message priority, publish little measurement, and in one widely used case lacked effective confidentiality and authenticity [1] (Chapter 2).

1.3 Motivation

Rescue is time-critical: survival depends on how fast rescuers learn that someone is there, where and in what condition. A system that makes one person's connectivity serve everyone and lets an SOS overtake routine traffic at every hop shortens that time. Beneficiaries are trapped occupants, rescue teams, and institutions (hospitals, campuses, hostels, factories, stadiums) that could pre-deploy it. The problem also exercises graph algorithms, concurrency, networking, cryptography and databases together and can be evaluated under controlled failure.

1.4 Objectives

O1 *Self-organising mesh*: neighbours found automatically, every node learns the whole graph (convergence $\leq 15\,\mathrm{s}$). O2 *Weighted routing* that beats hop count on identical scenarios. O3 *Priority*: SOS overtakes routine traffic at every hop ($\geq 3\times$ lower median latency under load). O4 *Reliability*: $\geq 90\,\%$ delivery on 40 nodes at 20 % loss and recovery within 10 s of a relay failure. O5 *Store-and-forward*: no route means retained, never dropped (100 % flushed). O6 *Security*: confidential, tamper-evident payloads; forged SOS rejected. O7 *Evidence*: every metric is a query over per-node logs. O8 *Integration*: web front end, REST/WebSocket back end and engine running on several devices.

1.5 Scope of the Project

Included: discovery with a neighbour table; a binary wire protocol; link-state gossip; Dijkstra over a link-quality cost; per-hop priority queue; TTL, dedup, ACK/retry with re-route, heartbeat; roles NORMAL, GATEWAY, RESCUE; bounded store-and-forward; AES-256-GCM and Ed25519; per-node SQLite log; `asyncio` concurrency; simulator and metrics; survivor and rescue interfaces. **Excluded:** real Bluetooth LE / Wi-Fi Direct transport, NAT traversal, voice/video/files, scale beyond about 50 nodes, machine learning, a native mobile app. **Users:** trapped user, relay user, gateway operator, rescue coordinator. **Environment and scale:** laptops running Python 3.10 on a shared

network; 5 live devices and up to 65 simulated nodes. The following statement is repeated verbatim because it is a design decision, not an omission:

> *The mesh currently runs over LAN sockets, which stand in for Bluetooth Low Energy or Wi-Fi Direct. On LAN we restrict each node's visible neighbour set to emulate limited radio range. This lets us develop and measure the routing, prioritisation and reliability logic without radio hardware. Discovery, routing, reliability and encryption are all transport-agnostic — replacing LAN sockets with a radio transport does not change any layer above it.*

1.6 Proposed Solution

Each device runs one node (a Python process whose `asyncio` loop runs discovery, link-state gossip, receive, send, store-and-forward and an API server). A message leaves through one of three exits: a **gateway** with an outside link (reaching any gateway counts as arrival); a **rescue** device that comes into range (every pending SOS flows in); or **store-and-forward**, where a node with no route keeps the message until one appears, so a person walking out of the building physically carries others' messages. Paths are chosen by Dijkstra over a graph whose edge weights penalise weak and failing links (Chapter 5).

1.7 Novelty and Originality

A **priority-aware, link-quality-weighted mesh with a delay-tolerant fallback, evaluated empirically**: SOS priority re-applied at every hop; cost derived from measured HELLO reception and from ACK failures, applied immediately; one transport interface so the same engine runs on a simulator, on loopback and on separate laptops; and every number is a query over per-node logs, with weighted and hop-count routing compared on identical seeds.

1.8 Feasibility Analysis

Technical: mature open-source components; the main risk (debugging routing over live sockets) was removed by building the simulator first. **Economic:** existing laptops, open-source software, no server, so there is no recurring cost. **Operational:** one command starts a node; the survivor screen has one prominent control. **Time/resources:** three members, one semester, about 336 person-hours, delivered in four tagged phases.

1.9 Social Relevance and Practical Applications

The realistic adoption path is *institutional pre-deployment*, not consumer download: hospitals, campuses, hostels, factories and stadiums install it on staff and resident devices as safety compliance; rescue and civil-defence teams carry it as standard kit so the rescue exit works even if no civilian has it; and it can ship as a module inside an app people already have. The benefit is faster location of survivors and coordination inside a stricken building at no infrastructure cost.

1.10 Success Criteria and Key Performance Indicators

The targets of Table 2 are the quantified requirements of the SRS, fixed before testing; Section 9.10 validates them against measured data.

Table 2: Key performance indicators

KPI	Definition	Target	Measured by
K1 Delivery	Delivered ÷ sent, 40 nodes, 20 % loss	$\geq 90\,\%$	seeded simulator runs
K2 Convergence	Every node's graph holds every node	$\leq 15\,\mathrm{s}$	simulator, live
K3 Recovery	Relay death until a route avoiding it	$\leq 10\,\mathrm{s}$	50 trials, live
K4 SOS advantage	Median routine ÷ median SOS latency under load	$\geq 3\times$	burst scenario
K5 Store-and-forward	Retained messages flushed on return	100 %	simulator, live
K6 Weighted vs hop count	Delivery on identical seeds	weighted better	paired runs
K7 Scale	Nodes within the targets	≥ 50	sweep 10–65 nodes
K8 UI freshness	Event to screen	$\leq 2\,\mathrm{s}$	API time, poll interval
K9 Correctness	Tests passed	100 %	repeated runs
K10 Security	No relay decryption; forged SOS and tampering rejected	all hold	automated tests

2 Literature Review and Study of Existing Systems

2.1 Study of Existing Systems

Offline messengers. *Bridgefy* is a Bluetooth mesh messenger used in large protests; its security analysis is Study 14 in Table 3. *FireChat* gave phone-to-phone messaging without a tower and is no longer maintained. *Briar* has no central server: online it relays through Tor, offline it synchronises directly between devices over Bluetooth or Wi-Fi with end-to-end encryption, so delivery happens when two devices connect [19]. **Dedicated radios.** *goTenna Mesh* uses its own paired radios; *Meshtastic* runs on LoRa with *managed flood routing*: every radio rebroadcasts up to a hop limit but suppresses its rebroadcast if a neighbour is already repeating it [20]. **Research and humanitarian systems.** The *Serval Project* builds phone meshes for disaster settings (first with B.A.T.M.A.N., later its own routing) with *Rhizome*, a store-and-forward data layer [21]; the *DTN architecture* provides store-carry-forward delivery across intermittent links [12, 13]. In the tables, "n/d" means not documented in the sources consulted.

2.2 Review of Research Papers and Technical Resources

Fifteen papers, standards and technical sources were studied; Table 3 records for each its technique, what it shows, and its relevance to and limits for this project.

Table 3: Fifteen studies: technique, contribution and relevance to EcoMesh

#	Source	Technique and contribution	Relevance and limitation
1	DSDV, Perkins and Bhagwat 1994 [2]	Proactive distance vector for mobile computers; destination sequence numbers prevent routing loops; periodic and triggered table updates	Routes are ready before the first emergency packet; hop-count metric, and update overhead grows in dynamic networks
2	DSR, RFC 4728 [3]	On-demand source routing: the complete route travels in every packet; route discovery floods a request, route maintenance reports broken links	Carrying the traversed path gives loop detection and "relayed via n"; hop-count shortest routes, no retention when discovery fails
3	AODV, RFC 3561 [4]	On-demand distance vector: route request flooded, reply returned on the reverse path, sequence numbers for freshness, route-error messages	Suits low traffic; hop-count metric, no long-term retention, no priority handling
4	OLSR, RFC 3626 [5]	Proactive link state; multipoint relays reduce the retransmissions needed to flood topology information	Closest protocol family to EcoMesh (every node learns the graph and computes paths locally); hop count by default
5	OSPF v2, RFC 2328 [6]	Link-state advertisements with sequence numbers flooded reliably; every router runs Dijkstra on its database; configurable interface cost	EcoMesh reuses the structure (sequence-numbered flooding, Dijkstra) but derives cost from measured links; built for stable wired networks
6	Babel, RFC 8966 [7]	Loop-free distance vector that is metric-agnostic and designed for wired and wireless meshes	Shows that the metric of a wireless mesh must be a pluggable, first-class idea; routing only, no reliability layer
7	ETX, De Couto et al. 2003 [8]	Path metric from the measured forward and reverse delivery ratio of probe packets; testbed shows minimum-hop paths use long, lossy links	Strongest justification for our cost function; we use the HELLO reception ratio plus an immediate ACK-failure penalty
8	Epidemic routing, Vahdat and Becker 2000 [9]	Replicate a message to every encountered node; exchange of summary vectors	High delivery but high buffer and bandwidth use; we keep a bounded queue instead

#	Source	Technique and contribution	Relevance and limitation
9	Spray and Wait, Spyropoulos et al. 2005 [10]	Create a limited number of copies, then wait for one to meet the destination	Motivates bounded retention rather than unbounded replication
10	PRoPHET, Lindgren et al. 2003 [11]	Forward to an encountered node only if its encounter history gives a better delivery predictability	History steers forwarding; a building mesh is stationary, so measured link quality is enough
11	DTN architecture, Fall 2003; RFC 4838 [12, 13]	Store-carry-forward with persistent storage and custody transfer for networks without end-to-end paths	Theory behind our third exit (a person carries messages out); no emergency priority or link-quality routing
12	Opportunistic networking, Pelusi et al. 2006 [14]	Survey and taxonomy of data forwarding in disconnected mobile ad hoc networks	Places store-and-forward in the wider design space
13	Disaster-response ad hoc networks, Reina et al. 2015 [15]	Survey of MANET, VANET, DTN and sensor networks for disaster response; multihop ad hoc communication is a practical fallback	Confirms the problem framing and the need for evaluated, interoperable systems
14	Breaking Bridgefy, Albrecht et al. 2021 [1]	Practical attacks on a deployed mesh messenger: user tracking, no authenticity or effective confidentiality, shutdown of the network by one crafted message	Sets the threat model; justifies authenticated encryption and signatures from the start
15	NIST SP 800-38D; Ed25519; Dijkstra [16–18]	AES-GCM authenticated encryption; fast high-security signatures; shortest paths	The two primitives (through the vetted `cryptography` library) and the routing kernel we use

2.3 Comparative Analysis

2.4 Limitations of Existing Systems

Hop count is the default metric of the surveyed routing protocols, although measurement shows that it picks lossy links [8]. None prioritises an emergency message over routine traffic at every hop. Ad hoc routing drops a message when no route exists, whereas delay-tolerant systems retain it but ignore link quality and priority. Deployed offline messengers publish little or no measurement, and at least one had no effective confidentiality or authenticity [1].

2.5 Identified Research or Engineering Gap

Existing consumer meshes flood or use simple hop-count routing, offer no message prioritisation and publish little measurement; routing protocols ignore retention, and delay-tolerant systems ignore link quality and priority. **No surveyed system combines priority-aware, link-quality-weighted routing with a delay-tolerant fallback and is evaluated quantitatively under controlled failure.**

2.6 Proposed Improvements

EcoMesh routes over the cost $1+5(1-q)+3f$ (Section 5.2); re-applies SOS priority at every hop; keeps unroutable messages in a bounded queue that evicts the lowest priority first and never an SOS in favour of routine traffic; authenticates and encrypts relayed messages with vetted primitives; and measures all of it on a seeded simulator and on separate devices, including a paired weighted-versus-hop-count experiment.

Table 4: Existing offline-communication systems and routing protocols compared with EcoMesh (n/d = not documented in the sources consulted)

System	Infrastructure	Routing	Priority	Store-fwd	Encryption
Bridgefy	Bluetooth LE phones	n/d	n/d	n/d	none effective in analysed version [1]
FireChat	phone-to-phone; discontinued	n/d	n/d	n/d	n/d
Briar	none (Bluetooth/Wi-Fi, Tor)	direct device sync	n/d	on contact	end-to-end [19]
goTenna Mesh	dedicated radios	n/d	n/d	n/d	n/d
Meshtastic	LoRa radios	managed flood [20]	n/d	n/d	channel-level
Serval	Wi-Fi ad hoc phones	B.A.T.M.A.N., later own	n/d	yes (Rhizome) [21]	n/d
DTN	none	architecture only	bundle class of service	yes (custody)	separate specification
AODV, DSR, DSDV	none	hop count / shortest route	none	none or short	not in protocol
OLSR, OSPF, Babel	none	hop count / configured / pluggable	none	none	not in protocol
EcoMesh	**none**	**link state, weighted Dijkstra**	**3 levels, every hop**	**yes, bounded**	**AES-GCM + Ed25519**

3 Software Requirements Specification

3.1 Purpose and Scope of the SRS

This chapter condenses the IEEE 830 Software Requirements Specification delivered in Phase II (`docs/SRS_EcoMesh_Group20.pdf`). It states *what* EcoMesh must do, not how. The scope is a single deployment of cooperating nodes with ten system features (peer discovery, message composition, multi-hop forwarding, adaptive routing, priority handling, reliability, store-and-forward, message security, logging, rescue dashboard); no server exists in any operating mode.

3.2 Intended Users and Stakeholders

The *trapped user* originates an SOS or a message and needs one prominent control and no configuration. The *relay user* passively carries other people's traffic and must not be able to read it. The *gateway operator* holds the residual outside link. The *rescue coordinator* monitors the SOS queue, the topology and delivery statistics. The *administrator* assigns node identifiers and constrains who hears whom. The project guide and evaluators review scope, rigour and evidence.

3.3 Overall Product Description

Every node runs the same program. HELLO beacons build a neighbour table; link-state reports build a graph; Dijkstra selects next hops; a per-hop priority queue orders transmissions; ACK, retry, deduplication and store-and-forward give reliability; AES-256-GCM and Ed25519 give confidentiality and authenticity; and an embedded web server exposes the node to a browser. The operating environment is an IP network segment on which infrastructure services have failed; where multicast is blocked a configured peer list is used (DIS-7). Dependencies are the Python runtime and the `cryptography` library.

3.4 Functional Requirements

The 68 functional requirements are listed in Table 5. Each has a unique identifier, its text, and an acceptance criterion: the tests that must pass (and, for screens, a manual check). Each feature's input, processing and output are given in the feature's heading row.

Table 5: Functional requirements by feature (identifiers as in the SRS). Check: Auto = verified by automated tests, Auto+UI = automated tests plus a manual check of the screen, Partial = only partly implemented, Inspect = code inspection, Manual = manual check only; * marks a mapped test that failed once in repeated runs. The first mapped test is named and (+n) counts the others.

ID	Requirement (the system shall . . .)	Check	Acceptance criterion: tests that must pass
3.4.1 Peer discovery. *Input:* HELLO beacons (role, public key) from neighbours; silence from a known neighbour. *Processing:* Update the neighbour table; estimate link quality over a 20-HELLO window; prune after the timeout. *Output:* Peer-up and peer-down events and link quality passed to routing.			
DIS-1	transmit a discovery beacon at a fixed, configurable interval.	Auto	`test_hello_is_broadcast_on_start`
DIS-2	keep a neighbour table with node id, address, role, public key, last-seen time and measured link quality.	Auto	`test_update_reports_new_neighbour_once` (+3)
DIS-3	add a previously unknown node to the table on receipt of a valid beacon.	Auto	`test_neighbour_added_on_hello_received` (+1)
DIS-4	declare a neighbour unreachable after a configurable period of silence.	Auto	`test_neighbour_marked_dead_after_timeout` (+1)
DIS-5	compute link quality as the proportion of expected beacons received over a sliding window.	Auto	`test_perfect_reception_scores_one` (+2)
DIS-6	operate without manual configuration of peer addresses.	Auto	`test_message_to_discovered_neighbour_delivers_without_manual_next_hop` (+1)

ID	Requirement	Check	Acceptance criterion
DIS-7	support an optional configured seed list of peers where multicast is unavailable.	Auto	`test_visible_neighbours_filters_unlisted_senders` (+1)

3.4.2 Message composition and transmission. *Input:* Text and destination, or the SOS control (sustained press). *Processing:* Build a packet with a unique id and timestamp; encrypt and sign; queue. *Output:* DATA packet sent; delivery state shown to the user.

MSG-1	let a user compose and send a text message to a named node.	Auto	`test_send_with_ack_delivers_and_gets_acknowledged` (+1)
MSG-2	let a user address a message to the rescue service rather than a specific node.	Auto	`test_rescue_dst_routes_to_gateway_across_multiple_hops` (+1)
MSG-3	provide a distinct SOS control that requires a sustained press.	Auto+UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered`
MSG-4	assign a globally unique identifier to every message it originates.	Auto	`test_msg_id_defaults_to_uuid4_and_round_trips` (+1)
MSG-5	record the creation time of every message it originates.	Auto	`test_timestamp_is_uint64_milliseconds` (+1)
MSG-6	display the current delivery state of each originated message.	Auto+UI	`test_store_logs_sent_message_and_delivery_lifecycle` (+1)
MSG-7	display the number of intermediate nodes traversed by a delivered message.	Auto+UI*	`test_update_message_route_records_hops_and_path` (+1)

3.4.3 Multi-hop forwarding. *Input:* DATA or ACK packet from a neighbour. *Processing:* Validate; drop duplicates; check hop allowance and loop; append own id; queue by priority. *Output:* Packet forwarded, or dropped with a logged reason.

FWD-1	forward packets not addressed to it towards the destination.	Auto	`test_two_hop_relay_a_to_b_to_c` (+1)
FWD-2	decrement the remaining hop allowance of every packet it forwards.	Auto	`test_on_forward_hook_called_with_next_hop`
FWD-3	discard any packet whose hop allowance has reached zero.	Auto	`test_drops_when_ttl_expired`
FWD-4	append its own identifier to the traversal path of every forwarded packet.	Auto	`test_on_forward_hook_called_with_next_hop` (+1)
FWD-5	discard any packet whose traversal path already contains its own identifier.	Auto	`test_drops_on_loop_detected`
FWD-6	validate the structure of a packet before any forwarding decision.	Auto	`test_bad_magic_rejected` (+3)
FWD-7	be unable to read the payload of a packet it merely forwards.	Auto	`test_relay_never_sees_plaintext_payload`

3.4.4 Adaptive routing. *Input:* Link-state reports; a destination. *Processing:* Merge into the graph; Dijkstra over the weighted cost; resolve RESCUE___ to the nearest gateway or rescue node. *Output:* Least-cost next hop; routes recomputed on topology change.

RTE-1	periodically transmit its own neighbour list to adjacent nodes.	Auto	`test_send_report_reaches_each_neighbour` (+1)
RTE-2	construct a graph of the network from received link-state reports.	Auto	`test_receiving_report_merges_reported_edges_into_graph` (+1)
RTE-3	compute least-cost paths using Dijkstra's algorithm over the weighted graph.	Auto	`test_shortest_path_multi_hop_chain` (+1)
RTE-4	derive link cost from a base hop penalty, measured link quality and recent delivery failures.	Auto	`test_edge_cost_matches_section_8_2_formula` (+1)
RTE-5	recompute routes whenever the topology changes.	Auto	`test_record_failure_invalidates_cached_routes` (+2)
RTE-6	treat any node advertising the gateway or rescue role as a valid terminal for rescue-service messages.	Auto	`test_rescue_dst_resolves_to_nearest_gateway` (+2)
RTE-7	select the least-cost terminal when several are reachable.	Auto	`test_rescue_dst_picks_cheapest_of_multiple_gateways`
RTE-8	converge on a consistent topology view within a bounded interval after a change.	Auto	`test_report_reflooded_reaches_a_node_two_hops_away` (+1)

3.4.5 Priority handling. *Input:* Packets of different priority queued together. *Processing:* Priority queue ordered by priority, then age, applied at every hop. *Output:* SOS transmitted before routine traffic.

PRI-1	support three priority levels: SOS, NORMAL, STATUS.	Auto	`test_all_three_priorities_ordered_correctly` (+1)
PRI-2	keep outgoing messages in a priority queue rather than a FIFO queue.	Auto	`test_sos_pops_before_normal_regardless_of_insertion_order`
PRI-3	apply priority ordering at every hop, not only at the originator.	Auto	`test_sos_forwarded_before_already_queued_normal` (+1)
PRI-4	order messages of equal priority by creation time, oldest first.	Auto	`test_ties_broken_by_oldest_timestamp_first`
PRI-5	assign SOS priority to every message generated by the SOS control.	Auto+UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered`
PRI-6	display the priority level of each message in the interface.	Manual	*Priority badges are shown on the screens (manual check); no automated UI test.*

ID	Requirement	Check	Acceptance criterion
3.4.6 Reliability. *Input:* Delivered or unacknowledged packets; duplicates; missing beacons. *Processing:* ACK; timer; retry with a recomputed route; dedup set; failure detection. *Output:* ACK returned; bounded retries; duplicates dropped; failed neighbours removed.			
REL-1	carry a hop allowance in every packet and decrement it at each hop.	Auto	`test_drops_when_ttl_expired` (+2)
REL-2	remember recently observed message identifiers for a configurable period.	Auto	`test_entry_expires_after_window` (+1)
REL-3	discard any packet whose identifier is already remembered.	Auto	`test_duplicate_delivery_is_dropped` (+2)
REL-4	transmit an acknowledgement on successful delivery.	Auto	`test_send_with_ack_delivers_and_gets_acknowledged` (+1)
REL-5	retransmit an unacknowledged message after a configurable timeout.	Auto	`test_timeout_triggers_retry_and_resets_timer` (+1)
REL-6	recompute the route before each retransmission.	Auto	`test_record_failure_updates_graph_immediately_not_just_on_next_report` (+1)
REL-7	abandon delivery after a configurable number of failed attempts and mark the message accordingly.	Auto	`test_exhausting_max_retries_reports_failed_and_stops_tracking` (+1)
REL-8	detect node failure through absence of discovery beacons and update the graph.	Auto	`test_route_forgotten_when_neighbour_dies` (+2)
3.4.7 Store-and-forward. *Input:* A message with no route; a new route or neighbour. *Processing:* Bounded retention by count and age; periodic flush; lowest priority evicted first. *Output:* Message delivered when a path appears, or expired and logged.			
SNF-1	retain a message when route computation yields no path.	Auto	`test_send_without_route_queues_to_store_forward`
SNF-2	periodically re-attempt delivery of retained messages.	Auto	`test_store_forward_flushes_once_a_route_appears`
SNF-3	attempt delivery of retained messages immediately on detecting a new neighbour.	Partial	`test_store_forward_flushes_once_a_route_appears` *Retained messages are retried every flush interval (10 s default); an immediate trigger on neighbour arrival is not implemented.*
SNF-4	bound the retention queue by both message count and retention period.	Auto	`test_enqueue_over_the_bound_evicts_lowest_priority` (+2)
SNF-5	discard the lowest-priority retained messages first when the bound is reached.	Auto	`test_enqueue_over_the_bound_evicts_lowest_priority` (+2)
SNF-6	report the number of retained messages in the interface.	Manual	*The held-message count is shown on the screens (manual check); no automated UI test.*
3.4.8 Message security. *Input:* Payload and packet identity fields. *Processing:* AES-256-GCM encryption; Ed25519 signature; verification and decryption on receipt. *Output:* Tampered or forged packets rejected.			
SEC-1	encrypt message payloads using authenticated symmetric encryption.	Auto	`test_encrypt_decrypt_round_trip` (+2)
SEC-2	ensure intermediate nodes cannot recover the plaintext of messages they forward.	Auto	`test_relay_never_sees_plaintext_payload`
SEC-3	sign the packet header and traversal path with a digital signature scheme.	Auto	`test_signed_message_verifies_and_delivers_one_hop` (+2)
SEC-4	verify the signature of every received packet before delivery.	Auto	`test_forged_signature_is_rejected_not_delivered` (+1)
SEC-5	discard any packet failing signature verification or authentication.	Auto	`test_forged_signature_is_rejected_not_delivered` (+2)
SEC-6	distribute node public keys through discovery beacons.	Auto	`test_hello_includes_public_key_when_configured` (+1)
SEC-7	use established cryptographic libraries and not implement primitives directly.	Inspect	*Code inspection: crypto.py only calls the cryptography library (AESGCM, Ed25519).*
3.4.9 Logging and history. *Input:* Every message and network event. *Processing:* Write to the node's SQLite database. *Output:* History and delivery statistics available by query.			
LOG-1	maintain a persistent local record of messages sent, received and forwarded.	Auto	`test_store_logs_forward_on_middle_node` (+1)
LOG-2	record status, creation time, resolution time, hop count and path of each message.	Auto	`test_record_message_stores_path_as_comma_separated` (+2)
LOG-3	record network events: node arrival and failure, route change, duplicate suppression, hop-allowance expiry, retransmission, retention.	Auto	`test_store_logs_peer_up_and_down` (+2)
LOG-4	store plaintext content only where the node is the originator or destination.	Inspect	`test_store_logs_forward_on_middle_node` *Forwarded rows have body NULL (Table 8); asserted for the relay's log, otherwise by inspection.*
LOG-5	let the user view and filter the local message history.	Partial	`test_list_and_count_messages_filter_by_direction_and_status` *History and filter queries exist in the store; the device screens list recent messages but offer no filter control.*

ID	Requirement	Check	Acceptance criterion
LOG-6	support extraction of delivery statistics from the stored records.	Auto	`test_list_and_count_messages_filter_by_direction_and_status` (+1)
3.4.10 Rescue dashboard. *Input:* Node state and events. *Processing:* Compute topology, SOS queue and statistics from the graph and the logs. *Output:* Live graph; SOS queue by priority then age; statistics.			
DSH-1	render the mesh topology as a graph of nodes and active links.	Auto+UI	`test_snapshot_describes_a_converged_mesh`
DSH-2	update the topology view as nodes join and fail.	Auto+UI	`test_killing_a_node_reroutes_and_reviving_restores_it`
DSH-3	display outstanding emergency messages ordered by priority, then age.	Auto+UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered`
DSH-4	display the originating node and traversal path of each received message.	Auto+UI	`test_sos_reaches_the_rescue_queue_and_the_sender_sees_it_delivered` (+1)
DSH-5	display aggregate statistics: delivery ratio, mean hop count, nodes online.	Auto+UI	`test_snapshot_describes_a_converged_mesh`
DSH-6	receive updates without the coordinator refreshing the view.	Auto+UI	`test_websocket_streams_a_hello_snapshot_then_events`

3.5 Non-Functional Requirements

- **Performance:** at least 90 % delivery on a forty-node network with 20 % packet loss; convergence of the topology view within 15 s; delivery resumed within 10 s of a node failure; an SOS measurably faster than routine traffic under load.
- **Scalability:** at least fifty nodes without degradation beyond these targets and without configuration changes to existing nodes.
- **Availability and reliability:** no single point of failure; correct operation under packet loss, node failure and partition; no message discarded while retention capacity remains.
- **Security and privacy:** payloads encrypted; relays cannot read them; tampering detected; a forged SOS rejected; every packet field validated; private keys never transmitted; plaintext stored only on the originator and destination.
- **Usability and accessibility:** the emergency action is one prominent control requiring a sustained press; no end-user configuration; plain-language status text and a state change visible within 2 s.
- **Maintainability, portability and resources:** independent modules for each concern; the engine is independent of the link technology, so another transport replaces the current one unchanged; one process on one event-loop thread with an embedded database and bounded queues (measured in Chapter 8).

3.6 Hardware and Software Requirements

Hardware: laptops (Windows 11 used for development and testing) on a Wi-Fi network or phone hotspot; no server. **Software:** Python 3.10 with `asyncio`; `cryptography` 42 (AES-256-GCM, Ed25519); SQLite in WAL mode; FastAPI, uvicorn, pydantic and `websockets` for the API; React 18, Vite 6, Tailwind 4 and d3-force for the web application (with a manifest and service worker so it installs as a PWA); pytest, pytest-asyncio and httpx for testing; Git and GitHub for version control.

3.7 Constraints and Assumptions

Constraints: one semester (phases from 17 August to 2 November 2026); three members; no radio hardware, so the radio link is emulated by a neighbour-restricted LAN; scale about 50 nodes; no machine learning and no native mobile application. **Assumptions:** the software is installed before an incident; every device is within range of another, directly or through intermediaries; batteries last the incident; at least one gateway or an arriving rescue node exists for messages to leave the mesh; node identifiers are unique and assigned in advance.

3.8 Use Cases and User Stories

UC1: as a trapped user I hold the SOS control so that rescuers learn I need help; the system sends the highest-priority message and shows its live status. **UC2:** as a trapped user with no route I want my message kept; it is retained, flushed when a path appears, and the screen says it is held. **UC3:** as a relay user I want to help without reading others' messages; the node forwards automatically and the payload is opaque to it. **UC4:** as a gateway operator I want to be an exit point; the node advertises the gateway role. **UC5:** as a rescue coordinator I want the SOS queue ordered by priority then age, with route and age, and the live topology. **UC6:** as an administrator I want to restrict who hears whom through a topology file. The use case diagram is Figure 4.

4 System Architecture and Design

4.1 Overall System Architecture

A node is one operating-system process. Inside it `asyncio` runs discovery (HELLO every 3 s, timeout 10 s), link-state gossip (every 5 s), the receive path, the sender (priority queue), the store-and-forward flush (every 10 s) and the API server on one event-loop thread. Every activity reaches the network only through the `Transport` interface (Figure 1). Shared mutable state is the neighbour table, the mesh graph and the pending-ACK map (plus the dedup set and the retention queue, each owned by one component); on one thread there are no locks and no data races by construction.

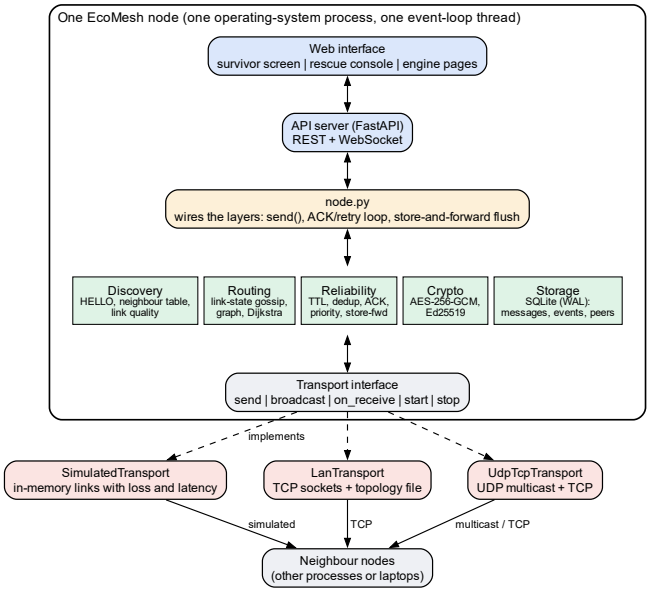


Figure 1: Block diagram: one EcoMesh node and its three interchangeable transports

4.2 Component and Module Description

Table 6 gives each module's purpose, input and output, and dependencies. **Wire protocol.** A fixed big-endian header (Table 7) is followed by the traversed path, the payload and an optional 64-byte signature. The path is carried on the wire for loop detection and so that the interface can show "relayed via n nodes". HELLO is a one-hop shout, LINK_STATE is unicast to neighbours, and DATA and ACK use TCP per hop, so each hop is ordered and reliable though the end-to-end path is not.

4.3 Architecture Selection and Justification

Layered and transport-agnostic. One boundary separates "how bytes move" from everything else. It gives a deterministic simulator, socket-free unit tests and an honest answer to the radio objection (a BLE transport is a fourth implementation, not a rewrite). **Single-threaded `asyncio` rather than threads.** Four activities share three structures; on one loop there are no locks or races, so correctness does not rest on lock discipline. **Peer-to-peer rather than client–server.** A server is a single point of failure and the very infrastructure being argued against. **Link state rather than flooding or on-demand discovery.** Flooding collapses under load and on-demand discovery delays the first emergency packet. **Per-node SQLite (WAL).** File-based, no server, and the API can read while the engine writes.

Table 6: Modules, purpose (input → output) and dependencies

Module	Purpose: input → output	Depends on
`packet.py`	pack, parse and validate the 52-byte header, path, payload and signature: bytes ↔ `Packet`	–
`transport*.py`, `lan_transport.py`	the transport boundary and its simulated, multicast/TCP and multi-device implementations: bytes between neighbours	`packet`
`discovery.py`	HELLOs → neighbour table, peer-up and peer-down events, link quality	`transport`
`relay.py`	received packets → deliver, forward in priority order, or drop with a reason	`reliability`
`routing.py`	link-state reports → mesh graph, least-cost next hop, `RESCUE__` terminal	`discovery`
`reliability.py`	packets → dedup decision, priority order, ACK timers, retained messages	`packet`
`crypto.py`	payload and identity bytes → ciphertext, signature, verification result	`cryptography`
`store.py`	messages, events and peers → SQLite rows and query results	`sqlite3`
`node.py`	wires every layer; `send()`, ACK/retry loop, flush loop	all above
`sim/`, `api/`	scenario scripts → metrics; REST/WebSocket server	`node`
`ui/`, `story/`	React application; one-device program with survivor and rescue screens	`api`, `node`

Table 7: The 52-byte packet header

Off.	Size	Field	Meaning	Off.	Size	Field	Meaning
0	2	magic	`0x4D45` “ME”	24	8	src	node id, ASCII
2	1	version	1	32	8	dst	node id / `RESCUE__`
3	1	type	HELLO, LINK_STATE, DATA, ACK	40	8	timestamp	Unix ms
4	1	priority	0 SOS, 1 NORMAL, 2 STATUS	48	2	payload_len	bytes
5	1	ttl	per-hop decrement (10)	50	1	path_len	ids in path
6	1	flags	enc., needs_ack, s-f, signed	51	1	attempt	retry counter
7	1	hop_count	per-hop increment	8	16	msg_id	UUIDv4

4.4 Data Flow Diagram (DFD)

Figure 2 is the context-level DFD: the system boundary contains one node and no server. Figure 3 is the level-1 decomposition into six processes (discover, build graph, receive, forward, originate/retry/retain, log and serve) and five data stores.

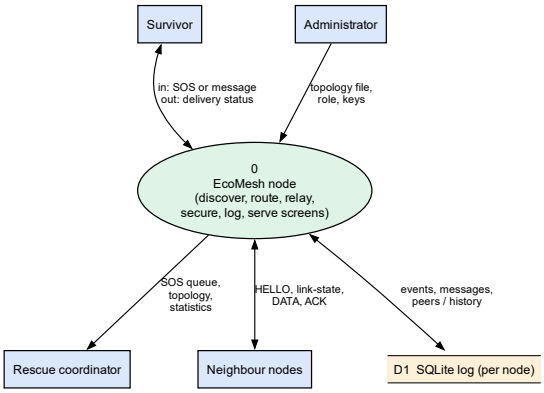


Figure 2: Level-0 (context) data flow diagram

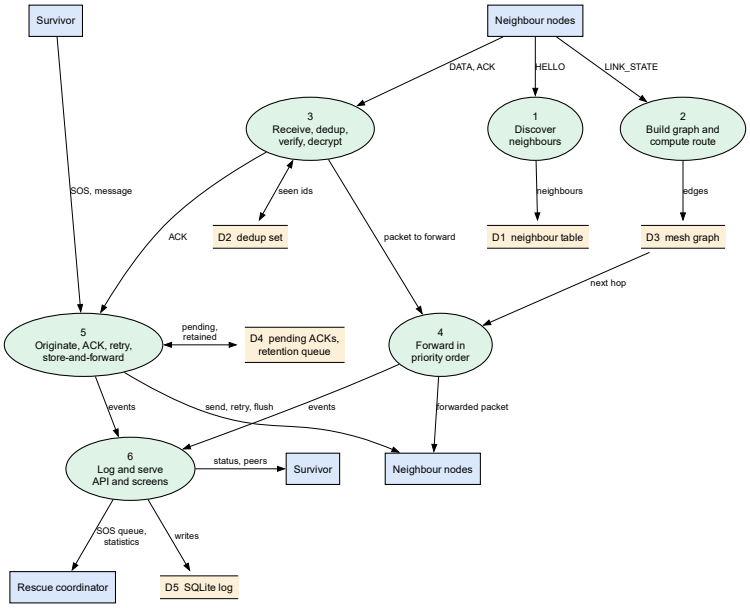


Figure 3: Level-1 data flow diagram of one node

4.5 UML Diagrams

Use case diagram. The actors are the trapped user, the passive relay user, the gateway node, the rescue coordinator and the administrator. Raising an SOS includes sending a message; holding and flushing undelivered messages extends sending when no route exists. **Class diagram.** `Node` owns the discovery, gossip, routing, relay, ACK-tracking, retention and storage components (filled diamonds); the router and gossip share one `MeshGraph`; the relay owns its priority queue and uses a dedup set;

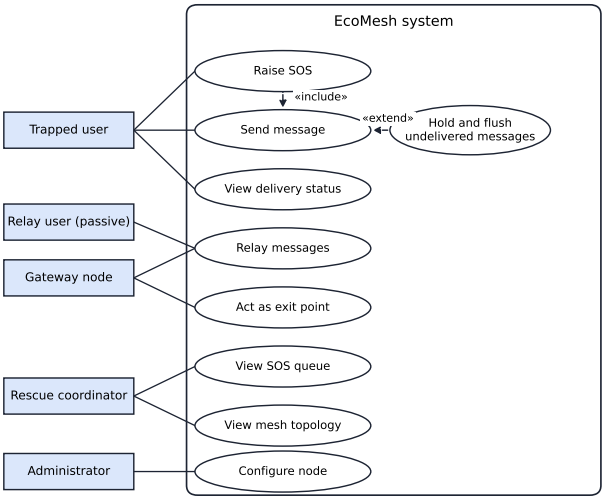


Figure 4: Use case diagram

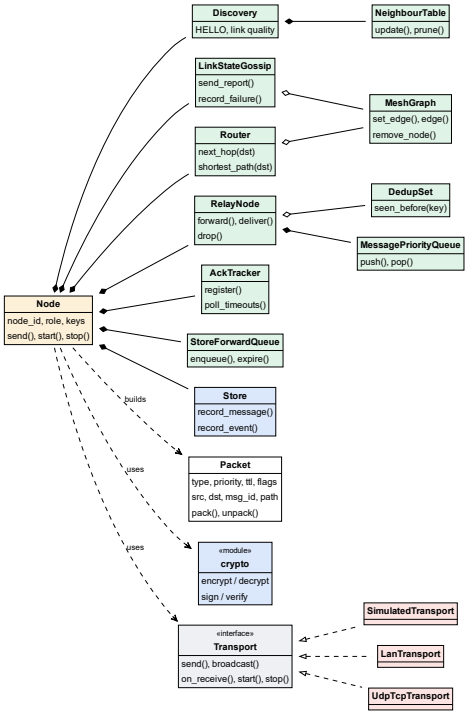


Figure 5: Class diagram: `Node` as the composition root

three classes implement the `Transport` interface. **Sequence diagram.** An SOS is created, encrypted

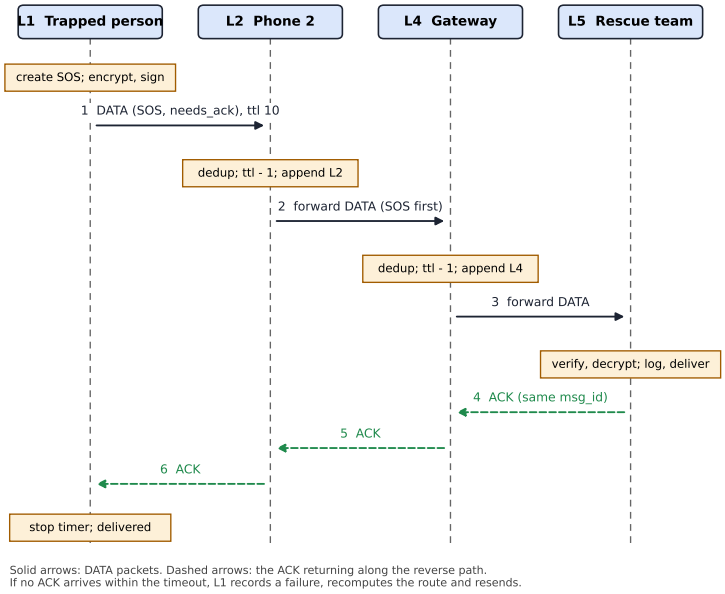


Figure 6: Sequence diagram: an SOS relayed hop by hop with the ACK returning along the reverse path

and signed at L1, forwarded through L2 and the gateway (each relay applies dedup, decrements the hop allowance and appends its own id) and delivered at L5, whose ACK returns along the reverse path. **Activity diagram.** A packet is validated and checked for duplicates; if it is for this node its

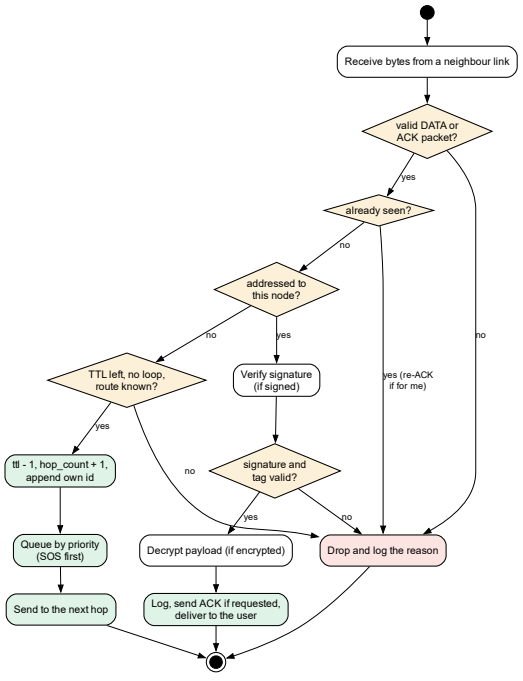


Figure 7: Activity diagram: the packet receive path

signature is verified, its payload decrypted, and it is logged, acknowledged and delivered; otherwise the hop allowance, loop and route checks decide between dropping it and re-queuing it by priority. **State**

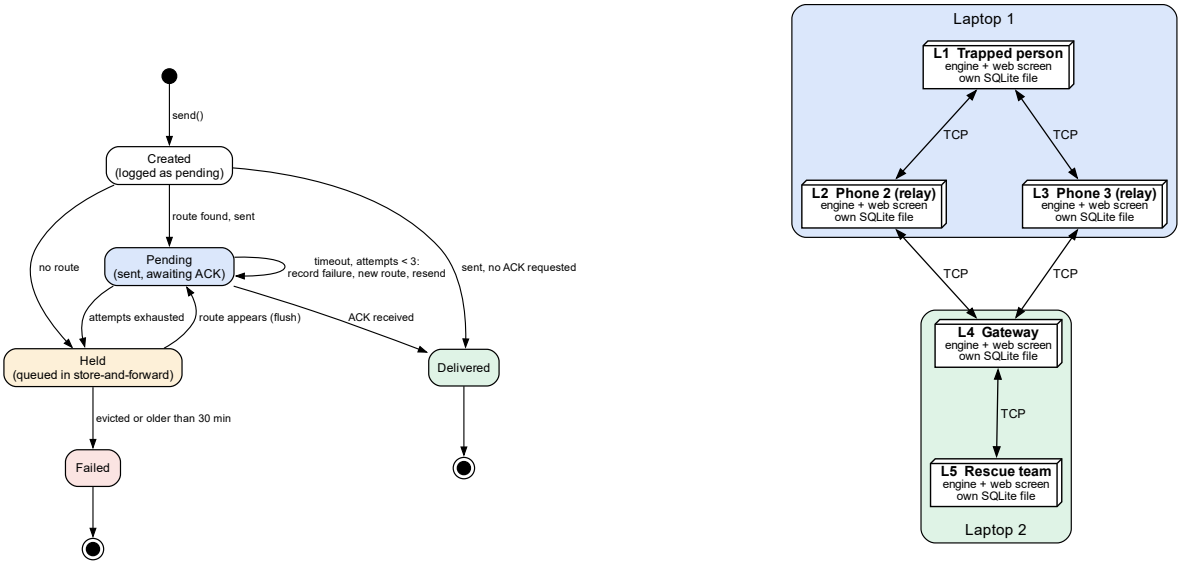


Figure 8: State machine of the message lifecycle (left) and deployment diagram of the live configuration (right)

machine. A message is created, then pending (sent, awaiting ACK) or held (no route), and ends delivered or failed; the states are the `status` values stored in the database. **Deployment diagram.** Five node processes on two laptops, each with its own engine, web screen and SQLite file, joined only by TCP links where the topology file lists them; there is no server (Section 7.7).

4.6 Database Design

Each node keeps its own SQLite database (WAL mode, plain `sqlite3` and raw SQL: three tables, so an ORM would add indirection for nothing). Every metric is a query over it (SRS 6.1). **Rela-**

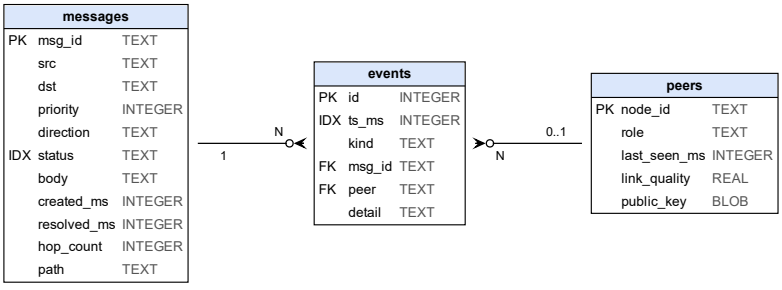


Figure 9: Entity–relationship diagram of the per-node database (PK primary key, IDX secondary index, FK logical foreign key)

tional schema. `messages`(`msg_id` PK, `src`, `dst`, `priority`, `direction` in sent/received/forwarded, `status` in pending/delivered/failed/queued_sf, `body`, `created_ms`, `resolved_ms`, `hop_count`, `path`); `events`(`id` PK, `ts_ms`, `kind`, `msg_id`, `peer`, `detail`); `peers`(`node_id` PK, `role`, `last_seen_ms`, `link_quality`, `public_key`). **Keys and integrity constraints.** `msg_id` is a UUID and is the

join key between a node's messages and events; `messages.body` holds plaintext only if the node is the source or destination; writes to `messages` are upserts, so a message moves pending → queued_sf → delivered without duplicate rows; `events.msg_id` and `events.peer` are logical, not declared, foreign keys because a relay logs events for messages it never stores and for peers that may have expired. **Normalisation.** Third normal form except `messages.path`, a comma-separated list kept denormalised because it is always read and written whole. **Indexing.** `idx_events_ts` serves "events since t" for the live feed; `idx_msg_status` serves the retained-message counter. **Sample records** from a real run are in Table 8; the relay's row carries no body.

Table 8: Sample rows of `messages` for one SOS from A to C in a real run: each node holds its own view of the same message id

DB	msg_id	src	dst	prio	direction	status	body	hops	path
A	8e859198...	A	C	0	sent	delivered	trapped on floor 3	1	B
B	8e859198...	A	C	0	forwarded	delivered	NULL	1	B
C	8e859198...	A	C	0	received	delivered	trapped on floor 3	1	B

4.7 Security and Error Handling Design

Authentication. Nodes have no user accounts; a node's identity is its Ed25519 key pair, distributed in HELLO and link-state reports, and every signed packet is verified against the sender's known key. An unknown signer is rejected. **Authorization.** Authority follows the node role: only GATEWAY and RESCUE nodes are valid terminals for `RESCUE__`; any node may relay. The relay path never decrypts or stores content it does not own, and a relay that does not hold the session key sees only ciphertext; because one pre-shared key serves the whole mesh, a node that *does* hold it could read what it forwards, which per-pair key agreement would remove (Section 9.11). **Input validation.** Every packet field is range-checked before use; a violation raises `PacketError` and the packet is dropped, never forwarded. **Exception handling.** A bad packet is dropped and logged and never raised into the event loop; crypto failures collapse into one exception type and `verify` returns `False` instead of raising. **Logging.** Each drop has a named event kind (`drop_ttl`, `drop_dup`, `drop_loop`, `drop_signature_invalid`, `drop_decrypt_failed`, ...) in the node's database. **Protection of sensitive data.** Payloads are AES-256-GCM ciphertext with a random 12-byte nonce; plaintext is stored only on the originator and destination; private keys stay in local memory and are never transmitted. The signature covers the identity fields (version, type, priority, flags, src, dst, msg_id, timestamp, payload) and not the whole header, because TTL, hop count and path change at every relay and a signature over them could never verify beyond the first hop; the retransmission counter is excluded so a retry keeps its identity. Table 9 maps threats to mechanisms.

Table 9: Threat model and countermeasures

Threat	**Mechanism**	**Requirement**
Relay reads a message	AES-256-GCM payload; a relay without the session key reads only `dst` and `ttl`	SEC-1, 2, FWD-7
Relay modifies a message	the GCM tag fails at the destination; packet dropped and logged	SEC-5
Relay forges an SOS	Ed25519 signature over the identity fields; public keys in HELLO and gossip	SEC-3, 4, 6
Replay of a captured packet	dedup set keyed on message id (300 s)	REL-3
Malformed or hostile bytes	strict validation; `PacketError`; drop	FWD-6
Endless circulation	TTL per hop; own id in the path is a loop	FWD-3, 5
Hand-rolled cryptography	only `cryptography`-library primitives	SEC-7

5 Algorithm Design and Data Structures

5.1 Identification of Critical Operations

Ten operations decide correctness and performance: (1) neighbour discovery and link-quality estimation; (2) link-state dissemination; (3) least-cost route computation; (4) the per-packet forwarding decision; (5) priority scheduling; (6) duplicate suppression; (7) send, acknowledge and retry; (8) bounded store-and-forward; (9) encryption and signing; (10) resolution of the symbolic destination `RESCUE__`. They correspond to Algorithms 1–10 of the Phase II *Design of Algorithms*; the pseudocode below reflects the implemented code, including the Phase 4 corrections.

5.2 Algorithm Design

A1 Discovery and link quality.

```
every 3 s: broadcast HELLO(role, public_key)                        // ttl = 1
on HELLO from p:  new <- table.update(p);  if new: on_peer_up(p)
    q <- hellos_received / hellos_expected   in a 20-slot window    // in [0,1]
    if |q - last_reported| >= 0.1: report q                         // avoids recomputing
    on noise
every 1 s: for p silent > 10 s: remove p; on_peer_down(p)
```

A new neighbour is scored over the HELLOs *due* so far, otherwise every fresh link would look lossy. (The Phase II write-up planned a 10-slot window; the code uses 20.)

A2 Link-state with re-flooding.

```
every 5 s: seq++; send report(origin=me, seq, [(p, q, failures, role, key) for p in
    neighbours]) to each neighbour
on report(origin, s, entries) from sender:
    if origin = me or s <= seq_seen[origin]: return                  // stale or own: stop
    seq_seen[origin] <- s;  replace origin's outgoing edges with entries;  learn roles and
    keys
    if changed: invalidate route cache
    forward report unchanged to every neighbour except origin and sender   // controlled re
    -flooding
```

Without re-flooding a node knows only its neighbours' neighbours; sequence numbers admit each report once per link, so flooding terminates.

A3 Dijkstra and the cost function.

```
cost(u->v) = 1 + 5 * (1 - q) + 3 * f        // f = ACK timeouts on the link in the last 60
    s
dist[src] <- 0; heap <- {(0, src)}
while heap: (d,u) <- pop_min; if u visited: continue; mark u
    for (v, e) in graph.neighbours(u): if d + cost(e) < dist[v]: dist[v] <- d + cost(e);
    prev[v] <- u; push
next_hop(dst) = path[1] of the cached shortest path, or none (=> store-and-forward)
```

Derivation of the coefficients. The base 1 is the per-hop penalty; it keeps every weight positive (Dijkstra's condition) and prefers fewer hops between equal links. The term $5(1-q)$ makes loss expensive: a link delivering 40 % of its HELLOs costs $1+5(0.6)=4.0$, more than two clean hops (2.0) or three (3.0), which is exactly where minimum-hop routing errs (Figure 10). The term $3f$ is steeper than the estimated-loss term because an ACK timeout is *observed* failure: one adds the cost of three extra clean hops, and it applies at once so the retry avoids the failing link.

A4 Packet handling.

```
on packet p from link sender s:
    if not parse_ok(p) or p.type not in {DATA, ACK}: drop
```

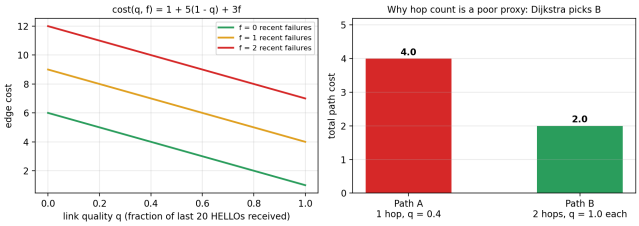


Figure 10: The cost function, and why hop count is a poor proxy for delivery

```
for_me <- p.dst = me or (p.dst = RESCUE__ and my role in {GATEWAY, RESCUE})
key <- (type, msg_id) if for_me else (type, msg_id, attempt)     // a retry crosses
relays that saw the original
if dedup.seen_before(key): drop(duplicate); re-ACK if for_me; return
if for_me: verify signature; decrypt; log; ACK if needs_ack; deliver
elif ttl <= 0: drop(ttl_expired)   elif me in path: drop(loop)   elif no next_hop: drop
(no_route)
else: ttl--; hop_count++; path.append(me); queue.push(p, priority)            // A5
```

The key includes the *type* because an ACK carries the id of the DATA it acknowledges (without it every ACK was dropped at the first relay the DATA had crossed, a bug found in integration), and the *attempt* at relays so a retransmission is forwarded; at the destination the attempt is excluded so the application sees each message once. A relay with no route drops the packet and logs `no_route`; only the originator retains (A8).

A5 Priority scheduling. `push` inserts (priority, timestamp, counter) into a binary heap and `pop` returns the highest priority, then the oldest; the counter prevents ties from comparing packets. Applied at *every* hop, an SOS overtakes routine traffic repeatedly along the route. **A6 Dedup.** A hash map of keys to first-seen time, pruned after 300 s. **A7 Send, ACK, retry.** Every 0.5 s the tracker is polled; a timed-out packet with fewer than 3 attempts gets a failure recorded on its link (cost +3), a recomputed route and a resend; after 3 attempts it moves to store-and-forward (Figure 11). **A8 Bounded store-and-forward.** `enqueue` evicts, when over 500 messages, the one with the largest (priority value, age), i.e. the lowest priority and oldest; `expire` drops anything older than 1800 s regardless of priority; the flush (every 10 s) sends each message whose destination now has a route. **A9 Security.** Send encrypts with a random nonce and signs the identity bytes; receive verifies against the sender's known key (an unknown signer is dropped) and then decrypts (a failed tag is dropped). **A10 `RESCUE__`.** One Dijkstra run, then the nearest reachable node whose role is GATEWAY or RESCUE; none reachable means store-and-forward (Figure 12).

5.3 Flowcharts

Figure 11 shows how a message is originated, acknowledged, retried and retained; Figure 12 shows how a next hop is chosen. The packet receive path is the activity diagram of Figure 7.

5.4 Data Structure Selection

5.5 Algorithm Selection and Justification

Flooding delivers, but its message count grows with the square of the node count and it collapses a mesh under load. Bellman–Ford tolerates negative weights, which never occur here, at $O(VE)$; A* needs a geometric heuristic that a building mesh lacks. Dijkstra with a binary heap is correct for non-negative weights and runs in $O((V + E) \log V)$. Hop count, the default metric of the surveyed protocols, picks lossy shortcuts (Chapter 8 measures the difference). A sorted list would make insertion $O(n)$ and a FIFO cannot express priority; a hash set gives $O(1)$ duplicate checks where a list gives

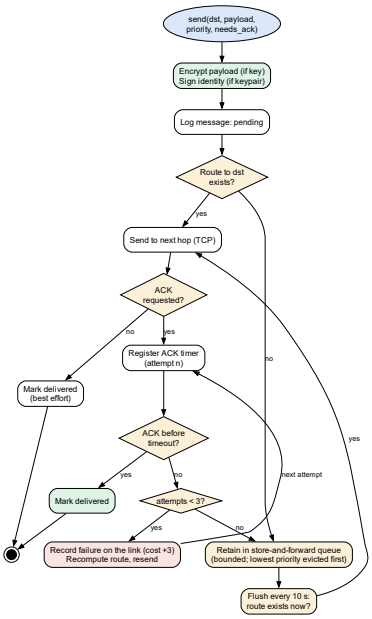


Figure 11: Flowchart: originate, acknowledge, retry and retain a message

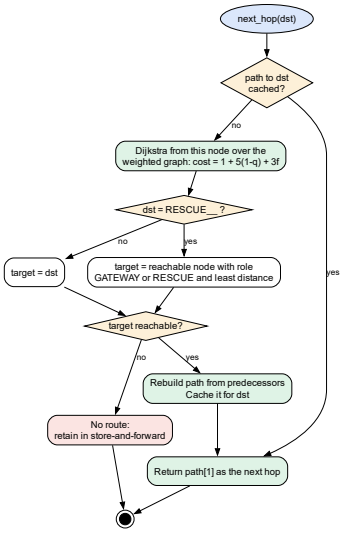


Figure 12: Flowchart: next-hop selection, including `RESCUE__`

Table 10: Data structures and why they were chosen

Structure and use	Reason
Adjacency dictionary of dictionaries: `MeshGraph`	$O(1)$ edge update and lookup, $O(\deg)$ iteration in Dijkstra; the mesh is sparse, so no matrix
Binary min-heap: Dijkstra frontier and the message priority queue	$O(\log n)$ insert and extract-min; ordering by (priority, timestamp, counter)
Hash map with timestamps: dedup set, neighbour table, route cache	$O(1)$ average membership; expiry by pruning
Fixed-capacity deque (20 slots) per neighbour	bounded sliding window for the link-quality estimate
Dictionary keyed by message id: ACK tracker, retention queue	$O(1)$ acknowledge and removal
Set: visited nodes in Dijkstra, reachable set	$O(1)$ membership

$O(n)$ on the hot path. The route cache is cleared on any topology change, which is simple and always correct.

5.6 Time Complexity Analysis

Table 11: Time complexity (V nodes, E links, n queued packets, q retained messages, D distinct destinations, $|N|$ neighbours)

Operation	Time
Dijkstra with the weighted cost	$O((V + E) \log V)$ in every case
Link-state merge on receiving a report	$O(\deg(v))$ plus $O(\|N\|)$ to re-flood
HELLO handling	$O(1)$ amortised
Priority enqueue / dequeue	$O(\log n)$; draining n packets $O(n \log n)$
Duplicate detection	$O(1)$ average
Loop check	$O(\|\text{path}\|)$, at most 10 here
Store-and-forward flush	$O(D\,(V + E) \log V + q)$ with the route cache; worst case $O(q\,(V + E) \log V)$
Eviction	$O(q)$ scan, only when over the bound
Packet parse and validate	$O(1)$ for the fixed 52-byte header

Dijkstra's bound holds for every input. The cache makes repeated lookups to a destination $O(1)$ until a topology change clears it.

5.7 Space Complexity Analysis

Per node the graph needs $O(V + E)$ (a few hundred entries at 50 nodes), the dedup set is proportional to the traffic in the 300 s window, the retention queue is bounded at 500 packets, and the neighbour windows hold $|N| \times 20$ timestamps. Dijkstra needs $O(V)$ extra for distances, predecessors and the heap. The SQLite log grows with the number of events and is the only unbounded structure; it lives on disk, not in memory.

5.8 Optimization Opportunities

Per-destination route-cache invalidation and incremental shortest paths instead of clearing the whole cache; multipoint relays to cut redundant copies of a report [5]; short retention at relays (a relay with no route now drops the packet); batched SQLite commits (each event commits, which is safe but limits the logging rate); and the timer changes that the measurements point to (Chapter 8).

6 Subject-Wise Engineering Concepts and APNI Leap Alignment

This chapter maps the Semester I–IV subjects that the project genuinely uses to the implementation; a subject the problem does not call for is marked not applicable, with the reason.

6.1 Software Engineering

Modularity: seven layers (discovery, transport, routing, reliability, crypto, storage, API) are separate modules joined only at `node.py`. *State management:* the node's mutable state is a small named set of structures, each owned by one component, and the message lifecycle is an explicit state machine (Figure 8) whose states are the stored `status` values. *Requirement traceability:* every requirement has an identifier and acceptance tests (Table 5). *Exception handling:* a bad input is dropped and logged and never raised into the event loop; cryptographic failures collapse to one exception type. *Version control:* Git with phase branches and Conventional Commits (Section 7.8).

6.2 Computer Networks

The communication architecture is peer-to-peer: each node is both host and router. The protocol is a custom binary format with a magic value, a version and a self-describing length, so a stream reader frames packets without a separate prefix (Table 7). HELLO is a one-hop broadcast, link-state reports are unicast to neighbours, and DATA and ACK use TCP per hop. `LanTransport` opens a connection per packet and prefixes it with the previous hop's identity, so flooding never echoes a report back to its sender; connect and drain are bounded by 1.5 s so a dead laptop fails fast. Errors are recovered by TCP per hop, by ACK, retry and route recomputation end to end, and by store-and-forward; failure detection uses missed HELLOs. *TCP between many clients and a central server is not applicable:* the project's requirement is the absence of a central server in every mode; the equivalent is many peers, each a server for its neighbours and a client of them, which the live deployment exercises.

6.3 Database Management Systems

Schema, primary keys, the deliberately denormalised `path` column, the two secondary indexes, upserts, logical foreign keys and WAL concurrency are in Section 4.6. *Transaction management:* every write is committed immediately, so each event is durable once logged (subject to SQLite's default synchronisation setting, which we did not tighten), and WAL lets the API read while the engine writes. *Query-driven metrics:* delivery ratio, hop histogram, per-priority latency (median and 95th percentile), retries, retained and flushed counts and the duplicate-drop ratio are queries over `messages` and `events`. *Distributed databases:* each node owns its file and there is no shared or replicated database; cross-node queries are assembled by the simulator or API reading each node's log.

6.4 Operating Systems

One node is one process; five processes ran live and communicated only over TCP and HTTP (interprocess communication). A single thread runs a cooperative `asyncio` event loop, so shared structures are touched only on that thread and there are no locks and no races by construction. Scheduling is *priority scheduling* (SOS, NORMAL, STATUS; ties oldest first) re-applied at every hop; Round-Robin was rejected because an emergency must not share a link fairly with routine traffic, and the scheduling metric is the SOS latency advantage measured in Chapter 8. I/O is non-blocking with timeouts, and memory is bounded (500 retained messages, 1800 s). On Windows with Python 3.10, `asyncio.wait_for` can swallow a task cancellation that arrives as the inner operation completes, which made `Node.stop()` hang intermittently; a cancellation-safe helper fixed it (Section 9.9).

6.5 Data Structures

Adjacency dictionaries, binary heaps, hash maps and sets, fixed-capacity deques and id-keyed dictionaries give efficient insertion, search, deletion and request handling (Table 10); the heap serves both Dijkstra's frontier and the priority queue.

6.6 Design and Analysis of Algorithms

Dijkstra's algorithm is correct here because every edge weight is at least 1, and it runs in $O((V + E) \log V)$ (Table 11). The cost function is a design contribution with derived coefficients (Section 5.2); link-state flooding terminates because sequence numbers admit each report once per link; and the weighted-versus-hop-count experiment (Chapter 8) is the empirical test of the design.

6.7 Cloud Computing

Not applicable by design: the system has no server, so there is no cloud deployment, hosted database or autoscaling, and no recurring infrastructure cost.

6.8 Mapping of Subjects to Project Implementation

Table 12: Subjects, concepts applied, modules and evidence

Subject	**Concept applied**	**Project module**	**Evidence**
Software Engineering	modularity, explicit state machine, error policy, traceability, version control	all; `node.py`	Table 5; Section 7.8
Computer Networks	binary protocol, framing, per-hop TCP, previous-hop identity, timeouts, failure detection	`packet.py`, `lan_transport.py`, `discovery.py`	`test_packet`, `test_lan_transport`; live test
DBMS	schema, keys, indexes, upserts, WAL, query-driven metrics	`store.py`, `api/metrics.py`	`test_store`; Section 4.6
Operating Systems	process per node, single-threaded event loop, priority scheduling, bounded memory	`node.py`, `reliability.py`	concurrency tests; priority scenario
Data Structures	heap, hash map and set, adjacency dictionary, deque	`routing.py`, `reliability.py`	`test_reliability`, `test_dijkstra`
Algorithms and Graph Theory	Dijkstra, cost derivation, link-state flooding, complexity	`routing.py`	weighted vs hop-count experiment
Cloud Computing	not applicable (no server by design)	–	–

7 Implementation and Module-Wise Coding

7.1 Development Environment

The system is written in Python 3.10 with `asyncio`, the `cryptography` library and SQLite; the API uses FastAPI, uvicorn and pydantic; the web application uses React 18, Vite and Tailwind; testing uses pytest and pytest-asyncio; version control is Git on GitHub. Development and testing used Windows 11 laptops. The source is about 6 500 lines of Python (`mesh/`, `api/`, `sim/`, `story/`), 4 500 lines of tests and 3 100 lines of React.

7.2 Overall Implementation Strategy

The build order followed what blocks what. **Phase 1:** the 52-byte packet and the `Transport` interface, so every track could work against a stable boundary. **Phase 2:** discovery, single-hop relay and the simulated transport, so that routing and reliability could be tested in one process. **Phase 3:** routing, reliability, security and storage, wired into `Node`. **Phase 4:** the API and React application, the scenario runner and metrics, engine fixes found by integration, and the multi-device deployment. Building the simulator first meant bugs were found deterministically in-process, not through live sockets.

7.3 Module-Wise Implementation

Table 13 gives, for each module, its code reference (file and size), its purpose, input and output, processing logic, dependencies and test evidence.

Table 13: Modules: code reference, logic, dependencies and tests

Module (lines)	Purpose, input → output, processing, details	Depends on	Tests
`mesh/core/packet.py` (362)	wire format, `Packet` ↔ bytes. `struct` format asserted to be 52 bytes; every field range-checked; `unpack` rejects bad magic and truncation of each region; `remaining_length` frames stream reads; `identity_bytes` is the signed content	–	47 + 4 000 fuzz
transports (461)	bytes between neighbours: simulated (seeded loss, latency), multicast/TCP, and the multi-device `LanTransport` (Section 7.7)	`packet`	15 + 6
`discovery.py` (292)	HELLOs → neighbour table, peer events, link quality over a 20-slot window reported only when it moves by 0.1; `visible_neighbours` range emulation	`transport`	15 + 12
`relay.py` (153)	received packets → deliver, forward or drop (algorithm A4); one drain task over a priority queue	`reliability`	18 + 3
`routing.py` (546)	reports → graph, next hop. `MeshGraph`, `edge_cost`, sequence-numbered gossip with quality, failures, role and key, cached Dijkstra, `RESCUE__` terminals, hop-count switch for the comparison	`discovery`	14 + 14 + 33
`reliability.py` (240)	`DedupSet` (300 s), `MessagePriorityQueue`, `AckTracker` (3 attempts), `StoreForwardQueue` (500 messages, 1800 s)	`packet`	18 + 20
`crypto.py` (131)	AES-256-GCM with nonce prefix and associated data; Ed25519 sign and non-raising verify; one exception type	`cryptography`	19
`store.py` (258)	tables, WAL, upserts, event queries, `events_since`, listing and counting for metrics	`sqlite3`	23
`node.py` (513)	composition root: `send()` encrypts, signs, logs, dispatches; receive verifies, decrypts, logs, ACKs; ACK loop records the failure, recomputes, resends; flush loop expires and retries	all above	27
`sim/`, `api/lab.py`, `api/metrics.py` (3 000)	simulator with a virtual clock and seven scenarios; metrics read from node logs	`node`	5 + 12

7.4 Front-End Implementation

Two front ends use the same engine. The **engine web application** (`ui/`, a React PWA) has an overview, a six-beat live demo that drives the real API, a node view with a long-press SOS control, the rescue dashboard (d3-force graph, SOS queue, statistics), and pages for routing, reliability, protocol, security and the simulator; a manifest and service worker make it installable. The **device screens** (`story/node.html`) are one page that becomes a *survivor phone*, a *gateway* view or a *rescue console* by role. The survivor screen is built to be read under stress: a plain status line, need chips and a location field, an SOS button that must be held about 1.2 s (preventing accidental sends), a progress tracker with plain-language states, an explanation when a message is held, and nearby phones with signal strength. The rescue console shows SOS cards (sender, text, route, age, a NEW badge), counters, a network map highlighting the last message's path, a device list and an optional alert sound. Screens are shown in Section 10.2.

7.5 Back-End Implementation

The back end is FastAPI with a WebSocket. The engine server (`api/server.py`) exposes the simulation lab (snapshot, reset, kill and revive, link edits, bursts), a node's view, graph and route (weighted and hop-count), `send` and `sos`, the rescue view, statistics, one message's full story, the event log, a protocol inspector, a security playground and attack drill, background scenario jobs, and `/ws`, which streams every logged event. Each device server (`story/node.py`) exposes `/api/me` (everything its screen needs), `/api/sos` and `/api/power`. Request bodies are validated with pydantic (text length, numeric ranges), and engine errors map to clear HTTP errors. The API reads the same objects and database as the engine.

7.6 Database Implementation

The three tables of Section 4.6 are created with `CREATE TABLE IF NOT EXISTS`, WAL mode is set at connection time, and the two indexes are created with the schema. Queries used include retained messages by status (`SELECT * FROM messages WHERE status = ?`), a message's timeline (`SELECT * FROM events WHERE msg_id = ? ORDER BY ts_ms`), event counters (`SELECT COUNT(*) FROM events WHERE kind = ?`), and the upsert `INSERT ...ON CONFLICT(msg_id) DO UPDATE`. Every write commits immediately.

7.7 Communication and Concurrency Implementation

The original `UdpTcpTransport` could not carry a complete `Node`: it never learned a HELLO sender's address, and it reported the packet's *original* source instead of the previous hop, which breaks link-state flooding. `lan_transport.py` fixes both. A JSON **topology file** lists every device (role, host, port) and the links (who can hear whom); a device talks only to its configured neighbours, which is the range emulation of the honest statement and satisfies DIS-7, so no multicast is needed. Each connection starts with the previous hop's identity followed by the unmodified packet. Connect, drain and close are bounded by 1.5 s so a dead laptop fails fast. A **power switch** makes a device neither send nor hear, like a dead radio. `python -m story.node -id L1` starts one device (engine plus a small web server); five of them, started together on one machine or one per laptop, form the mesh, each with its own SQLite log, port and screen. In the two-laptop demonstration L1–L3 ran on one laptop and L4–L5 on the other, joined through a phone hotspot (Figure 8). All of a node's activities share one thread, and `asyncio.wait_for` was replaced by a cancellation-safe helper (Section 9.9). Recovery is layered: TCP per hop; ACK, retry and re-route end to end; store-and-forward when retries run out.

7.8 Git Repository Setup

Repository: `https://github.com/tanvisheth1234-wq/miniproject`. **Structure:** `mesh/core/` (engine), `api/`, `sim/`, `story/`, `ui/`, `tests/`, `docs/`, `config/`, with README, CONTRIBUTING and requirements from the first phase. **Branching strategy:** one branch per phase, each inheriting the previous and never merged back (`develop`, `develop_2`, `develop_3`, `develop_4`, `develop_5`). **Commit conventions:** Conventional Commits (`feat(routing): ...`, `fix(reliability): ...`). **Contribution of members:** Tanvi Sheth built the packet format, transports, the API, the React application and the scenario runner, and the Phase 4 engine fixes found by integration; Miti Shah built discovery, the relay, the node skeleton and the simulated transport in Phase 2 and, in Phase 3, Dijkstra pathfinding with gateway and rescue handling and route caching, the dedup set and priority queue, ACK tracking with retry and the store-and-forward queue, AES-256-GCM encryption with Ed25519 signing, and the SQLite log; Twissha Shah built the weighted cost function and link-state gossip, the wiring of routing, reliability, crypto and storage into the node, the multi-device program and device screens, the robustness and socket tests, and the evidence scripts. Nine of Miti's Phase 3 commits were made from Twissha's laptop and were first recorded under Twissha's Git account; they were re-attributed to Miti on 24 October 2026 with the agreement of all three members, and the correction is recorded in `docs/contribution-log.md`. **Evidence of development history:**

Table 14: Commits per team member, read from `git log` over all remote branches (49 commits)

Member	Commits	Share
Tanvi Sheth	19	39 %
Twissha Shah	18	37 %
Miti Shah	12	24 %

Conventional Commits. 44 of 49 commits (90 %) follow the `type(scope): subject` convention of `CONTRIBUTING.md`.

Branches: `main`, `develop`, `develop_2`, `develop_3`, `develop_4`, `develop_5`. **Tags:** `v0.3-phase3`.

7.9 Integration of Modules

`Node` registers one dispatcher on the transport that feeds each packet to discovery (HELLO), gossip (LINK_STATE) and relay (DATA, ACK); each ignores types it does not own. Callbacks connect the pieces: peer-up sets a one-hop route and a graph edge and logs the event; a topology change invalidates cached routes; relay hooks write the message and event log; the ACK loop reports failures to gossip so the cost reacts at once. The API reads the same objects and database, so nothing on screen is computed separately from the engine.

7.10 Implementation Challenges and Resolutions

The significant difficulties were: a single receive callback that let only one of discovery and relay hear packets; topology known only two hops out, with dead nodes still routable (fixed by sequence-numbered re-flooding and replacing a newer report's edges); signatures that failed after the first relay (fixed by signing identity fields only); ACKs and retries lost at relays (dedup now keyed on type, id and attempt); a hostile packet raising `UnicodeDecodeError`; a hang in `Node.stop()` on Windows; and a transport that could not run a full node (fixed by `LanTransport`). Each was found by an integration, fuzz or repeated-run test and is listed with its cause and retest in Table 21.

8 Scalability, Performance and Resource Management

Every number in this chapter is generated by scripts in `docs/report/evidence/` from stored measurements; none is typed by hand. Two kinds of measurement are never mixed: the *simulator* (real `Node` objects over an in-memory transport with seeded loss and latency on a virtual clock, so the specified 3 s/10 s/4 s timers run in a fraction of a second) and *live devices* (real processes, real TCP, real timers).

8.1 Scalability Requirements

The SRS requires operation at up to fifty nodes, at least 90 % delivery on a forty-node network with 20 % loss, convergence within 15 s and recovery within 10 s (Section 1.10). The live deployment is five devices; larger sizes exist only in the simulator and are labelled as such.

8.2 Workload and Load Model

Each scenario fixes a topology, a per-link loss rate, 5–40 ms link latency, 10–90 messages (a fraction of them SOS) spread over a window, a settling period and a failure script. Each runs on many seeds; a seed fixes loss and traffic, so runs reproduce and routing modes can be compared on identical seeds. The loss sweep varies per-link loss from 0 to 50 % on a 21-node mesh; the scale sweep varies the mesh from 10 to 65 nodes at 5 % and 20 % loss; the throughput test sends 100 to 2 000 simultaneous messages over a three-node line. The live run sent 30 sequential SOS, caused 8 relay failures and 4 gateway outages with 5 held messages each.

8.3 Resource Utilization

Simulator: CPU time and Python heap peak of the whole simulated mesh in one process (Table 18); the 65-node mesh used about 13 MB of heap. **Live devices:** working set and CPU of each real process, engine plus web server (Table 19): about 53 MB and 0.4–1.9 % of one core when idle. **Storage:** the SQLite file grows with logged events (one row per event) and is the only unbounded structure; retention is bounded at 500 messages. **Network:** data transmissions per delivered message are in Tables 15 and 18; HELLO and link-state control traffic is not logged per packet and is not included.

8.4 Concurrent Processing

A node is single-threaded and cooperative with no locks. On a three-node line with ACKs requested, 2441 messages per second were delivered at 1 000 simultaneous sends and 1561 per second at 2000 (all ACKed within 2.7 s), with a SQLite commit per event. The concurrency tests (Chapter 9) confirm exactly-once delivery of 200 simultaneous sends.

8.5 Scheduling and Synchronization Analysis

The scheduling policy is priority scheduling with ties broken oldest-first, re-applied at every hop. The scheduling metric is the latency advantage of an SOS over routine traffic on the same trip, measured in a deliberately congested scenario (a radio that sends one packet per 80 ms) because on an idle network there is nothing to overtake. No synchronisation primitive exists because shared state is touched only on the event-loop thread. Results are in Section 8.7.

8.6 Performance Measurement

Experimental setup and test environment: one Windows 11 laptop, Python 3.10; simulator runs used a process pool; live runs used five processes on the loopback interface with HELLO 1 s, neighbour

timeout 4 s, link-state 2 s, ACK timeout 3 s and 3 retries. **Workload:** as in Section 8.2. **Tools and method:** per-process memory and CPU from operating-system counters; latency by polling each device's own API every 20 ms (so it carries up to 20 ms of error); simulator metrics are queries over the nodes' SQLite logs; repeated seeded runs and trials as stated beside each result.

8.7 Performance Results

Reliability under loss.

Table 15: Reliability: delivery ratio as per-link loss increases (21-node grid, 8 seeded runs per point, ACK and retry enabled)

Per-link loss	Delivery (mean)	Delivery (worst run)	Median hops	Transmissions per delivered	Convergence (s)
0%	100.0 %	100%	1.7	5.8	5.2
10%	100.0 %	100%	2.1	10.8	5.2
20%	98.4 %	94%	1.9	15.5	9.6
30%	89.3 %	77%	1.8	19.9	16.0
40%	76.4 %	69%	1.5	25.0	20.2
50%	61.4 %	42%	1.1	29.5	–

Delivery stays at or above 98 % to 20 % loss and falls to 61 % at 50 %; each lost ACK costs a 4 s timeout, so median latency goes from tens of milliseconds at 0 % to seconds at 20–30 %.

Scenarios.

Table 16: Results of the seven simulator scenarios (weighted routing; delivery ratio shown as mean (min–max) over the runs)

Scenario	Nodes	Loss	Runs	Delivery	Hops	Conv. (s)	Recov. (s)	S&F flushed/queued
baseline_10	10	5%	20	100.0 % (100–100)	1	5.5	–	7/7
collapse_40	40	20%	50	92.6 % (76–100)	4	9.6	8.1	2959/3187
no_gateway	12	5%	20	100.0 % (100–100)	2	5.2	–	201/201
high_loss	20	35%	20	89.1 % (78–100)	2	17.7	–	1007/1048
chain_10hop	11	5%	20	100.0 % (100–100)	9	14.4	–	87/90
hopcount_vs_ weighted	5	45%	20	99.2 % (90–100)	2	5.2	–	–
priority_load	16	0%	20	100.0 % (100–100)	2	7.6	–	1356/1356

Hops = median hop count of delivered messages. Conv. = mean seconds until every node's graph contains every node. Recov. = mean seconds from relay death until the sender has a route avoiding it. S&F counts are event counts summed over all runs: a message re-queued after a failed retry is counted each time, and messages still held when a run ends are not flushed.

The 40-node collapse scenario (20 % loss, a relay on the active path killed at $t = 10$ s, the gateway killed at 25 s and revived at 60 s) delivered 92.6 % on average over 50 seeded runs (worst 76 %, best 100 %). The 35 %-loss scenario delivered 89.1 %, just under 90 %, outside the specified condition. **Weighted versus hop-count routing (Figure 13b).** In the designed topology the 2-hop shortcut

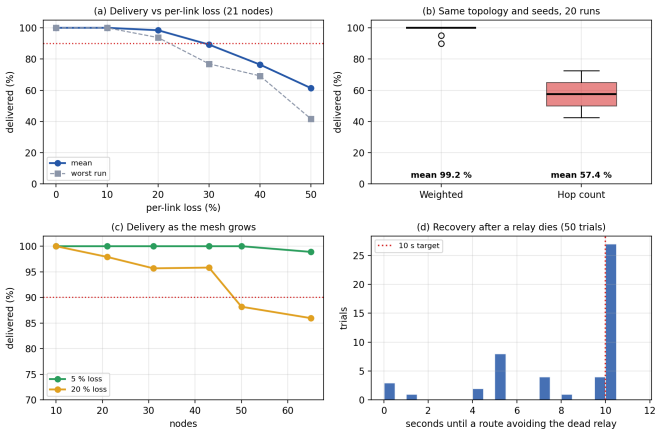


Figure 13: (a) Delivery against per-link loss. (b) Weighted versus hop-count routing on identical seeds. (c) Delivery as the mesh grows. (d) Recovery time after a relay death, 50 trials.

crosses a link with 45 % loss and the 3-hop path is clean; messages are best-effort, so a poor route shows up directly as loss. Weighted routing delivered 99.2 % and hop-count routing 57.4 %, and weighted won on 20 of 20 identical seeds. It is a *designed* topology that exposes hop count's weakness, so it shows that the cost function avoids a lossy shortcut, not that the same gap occurs on every mesh.

Priority under load.

Table 17: Latency by priority under load (16 nodes, one packet per 80 ms radio, 90 messages of which 15 % are SOS, all to the same destination; mean over 20 runs of the per-run statistic)

Priority	Median latency (ms)	95th percentile (ms)	Delivered per run
SOS	547	1034	9.5
NORMAL	2198	6145	49.2

Median SOS advantage (median NORMAL / median SOS): 4.4×. Only this scenario sends both priorities on the same trip, so only here is the comparison fair.

The congested scenario (16 nodes, 90 messages to the same destination, 15 % SOS) gives an SOS a median latency 4.4× lower than routine traffic. Only here do SOS and routine messages take the same trip, so only here is the comparison fair.

Scalability.

Table 18: Scalability, 10 to 65 simulated nodes (6 runs per size and loss rate)

Nodes	Links	Delivery at 5% loss	Delivery at 20% loss	Tx per delivered (5%)	Tx per delivered (20%)	Conv. max at 20% (s)	CPU (s, 5%)	Heap (MB, 5%)
10	13	100.0 %	100.0 %	5.2	7.8	10.2	3	1.1
21	32	100.0 %	97.9 %	7.7	15.7	15.2	12	2.3
31	50	100.0 %	95.7 %	10.2	23.4	10.2	29	4.0
43	72	100.0 %	95.8 %	12.7	24.6	15.2	52	6.9
50	85	100.0 %	88.2 %	14.4	33.3	10.2	69	8.5
65	113	98.9 %	86.0 %	20.8	39.3	15.2	126	12.9

Nodes and links include the gateway and rescue post added to the grid. Tx = hop-by-hop data transmissions. CPU and heap are for the whole simulated mesh in one process on a virtual clock, not the cost of one real device.

At 5 % loss delivery is 100 % to 50 nodes and 98.9 % at 65; at 20 % loss it holds above 95 % to about 43 nodes and falls to 88.2 % at 50 and 86.0 % at 65. Transmissions per delivered message grow roughly linearly with size (paths lengthen), not quadratically as flooding would.

Recovery and convergence. Both are quantised by design constants (Figure 13d). Convergence comes in multiples of the 5 s gossip interval (5.2, 10.2 or 15.2 s). Recovery splits in two: 23 of 50 trials recovered in under 10 s because ACK failures penalise the failing link at once; the other 27 recovered at 10.2–10.25 s, the 10 s heartbeat timeout plus the 0.25 s measurement step, because the sender did not learn of the death until its neighbour timed out.

Live devices.

Table 19: Live devices: 5 operating-system processes, real TCP sockets, real timers, on one laptop (loopback)

Measurement	Result
Process start → every device knows every device	3.8 s
SOS arrives at the rescue device (30/30 delivered)	median 47 ms, 95th pct 102 ms, max 281 ms
ACK back at the sender	median 63 ms, 95th pct 149 ms
Relay switched off → sender re-routes (8 trials)	median 4.0 s, max 4.5 s; every later SOS delivered
Gateway down, 5 SOS held; gateway back (4 trials)	100% flushed, median 4.5 s, max 6.1 s
Device API response (200 requests)	median 16 ms, 95th pct 31 ms
Per-process memory and CPU, idle	53–54 MB; 0.4–1.9% of one core
Per-process CPU while sending SOS (includes the harness polling every 20 ms)	up to 19% of one core

Timer settings: HELLO 1 s, neighbour timeout 4 s, link-state 2 s, ACK timeout 3 s. Latency is measured by polling each device's own API every 20 ms (up to 20 ms of error). Wireless delay and loss are not included.

8.8 Bottleneck Identification

Failure-detection latency (recovery is bounded by the 10 s heartbeat timeout); the 4 s ACK timeout under loss; the 5 s convergence granularity of the link-state interval; one SQLite commit per event; and wholesale route-cache invalidation.

8.9 Optimization and Improvement

No optimisation was applied after these measurements, so there is no before-and-after comparison to report. The data point to the changes with the best return: a neighbour timeout near 6–7 s so that recovery fits inside 10 s, an immediate link-state report on topology change, an adaptive ACK timeout from the measured round-trip time, and per-destination cache invalidation. Each is a one-constant or one-function change that can be re-measured with the same scripts.

8.10 Cloud Deployment and Resource Utilization

Not applicable: the system has no server and was not deployed on cloud infrastructure.

8.11 Scalability Limitations

The largest tested sizes are 65 simulated nodes and 5 live devices. *Measured:* 98.9 % delivery at 65 nodes with 5 % loss and 86.0 % with 20 % loss. *Not tested:* more than 65 nodes, wireless interference and mobility; capacity beyond the tested sizes is a projection, not a result. The live timings share one machine and exclude wireless delay.

9 Testing and Validation

9.1 Testing Strategy

Testing is layered, and each layer answers a different question: unit (does a module behave in isolation), integration (do modules work together in a node, between nodes, through the API and over real sockets), functional (does each SRS requirement hold), non-functional (reliability, scalability, performance, availability, security, usability), boundary and invalid-input, concurrency and stress, and live-device tests. The key design choice is the simulated transport: routing and reliability run on a deterministic in-memory network, so a failure reproduces from a seed. The suite has 312 automated test cases, run completely 3 times for the results below. Complete suite runs: 3; fully green: 2. Inconsistent test(s): `test_api::test_node_view_reports_peers_with_hop_distance`.

9.2 Unit Testing

Unit tests cover each module on its own: the packet format (47 tests: round trips, every field's range, null padding, truncation of each region, bad magic, signable and identity bytes, retransmission counter); the cost function and graph; Dijkstra including caching and `RESCUE__`; gossip encoding and merging; discovery and link quality; the dedup set, priority queue, ACK tracker and store-and-forward bounds; AES-GCM and Ed25519; the SQLite log; the relay; and the simulated transport.

9.3 Integration Testing

`test_node` (27 tests) runs discovery and relay on one transport, auto-relaying, role propagation, multi-hop origination via re-flooded reports, ACK, the whole message lifecycle in the database, encryption and signing end to end, retry exhaustion into retention, eviction and expiry. `test_ack_multihop` checks that the ACK returns over the data's path, that a lost ACK is re-sent and that a retry crosses relays that saw the original. `test_api` drives front end, back end and engine through FastAPI's test client, including the WebSocket: a converged snapshot, an SOS reaching the rescue queue, node views, kill and revive, the event stream, the protocol inspector and the security drill. `test_lan_transport` checks previous-hop identity, range emulation, the power switch, hostile bytes, a dead peer failing fast, and a full three-node mesh over TCP. The React interface and device screens have no automated browser tests; they were inspected by running them (Section 10.2).

9.4 Functional Testing

Table 5 lists every requirement with its acceptance criterion, the tests that must pass. 68 functional requirements: 52 verified by automated tests alone, 10 by automated tests plus a manual check of the screen, 2 partially implemented, 2 by code inspection, 2 by manual check only. All mapped tests passed in every complete run except one timing-sensitive test that failed once (Section 9.9). Two requirements are only partly met and are reported as such: SNF-3 (an immediate delivery attempt when a new neighbour appears; retained messages are instead retried every flush interval) and LOG-5 (the history filter exists in the store but the screens have no filter control).

9.5 Non-Functional Testing

Reliability: delivery at 0–50 % injected loss (Table 15). **Scalability:** 10 to 65 nodes at 5 % and 20 % loss (Table 18). **Performance:** latency by priority (Table 17), throughput, and live latency (Table 19). **Availability:** recovery over 50 simulated trials and 8 live failovers (Figure 13). **Security:** a relay without the session key never sees the plaintext (`test_relay_never_sees_plaintext_payload`); a tampered ciphertext or a wrong key is detected and dropped; a forged-signature SOS and an unknown signer are rejected; signatures verify across multi-hop relays; and the API's attack drill checks that

every attack is defended. **Usability:** the SOS control is a single hold of about 1.2 s on one prominent button with plain-language status; a timed study with participants was not carried out (Section 9.11).

9.6 Boundary and Invalid-Input Testing

Tests cover empty and header-only input; a bad magic value; truncated header, path, payload and signature; TTL out of range; a payload over the 16-bit length; a path that is too long; node ids over 8 bytes or not ASCII; a wrong-length signature; the signed flag without a signature; a wrong key size; ciphertext shorter than the nonce; an empty payload (valid); and a send to an unknown destination (held, not an exception). `test_robustness` also feeds 3 000 random byte strings and 1 000 bit-flipped valid packets to the parser and 200 garbage buffers to a running node, requiring only "parsed" or "rejected with `PacketError`" and that the node still delivers a normal message afterwards.

9.7 Concurrency and Stress Testing

200 simultaneous sends over two hops with ACKs: all 200 delivered exactly once and every ACK returned. Two senders sending 50 SOS and 50 routine messages simultaneously to one gateway: all 100 arrive intact. One packet replayed five times: delivered once, four duplicates dropped. Throughput up to 2 000 simultaneous sends (Section 8.7) and the congested priority scenario complete the stress tests. Because shared state lives on one thread, races and deadlocks cannot occur by construction; the tests confirm exactly-once delivery and stability under load.

9.8 Test Case Specification

Table 20: Representative test cases (every requirement's tests are in Table 5)

ID	Req.	Scenario, input and preconditions	Expected result	Actual result and evidence	Result
TC-01	RTE-3, 4	short path over a lossy link, long path over clean links	Dijkstra takes the clean path	as expected; `test_dijkstra`	Pass
TC-02	RTE-8	report from a node two hops away	re-flooded; graph complete beyond the neighbour	as expected; `test_link_state`	Pass
TC-03	PRI-3	SOS reaches a relay with routine packets queued	SOS forwarded first	as expected; `test_relay`	Pass
TC-04	REL-5, 6	ACK lost; sender times out	route recomputed, resent, destination sees it once	as expected; `test_ack_multihop`	Pass
TC-05	SNF-1, 2	no route; a route later appears	held, then flushed and delivered	as expected; live: 4 of 4 outages flushed (Table 19)	Pass
TC-06	SNF-5	retention queue full	lowest priority evicted; an SOS is never displaced	as expected; `test_ack_retry`	Pass
TC-07	SEC-5	forged-signature packet	rejected and logged	as expected; `test_node`	Pass
TC-08	FWD-6	3 000 random byte strings	only `PacketError`; node survives	one defect found and fixed (D4); `test_robustness`	Pass
TC-09	DIS-7	three nodes over real TCP, neighbour-restricted	routes learned; message and ACK delivered	as expected; `test_lan_transport`	Pass
TC-10	K1	40 nodes, 20 % loss, relay and gateway failures, 50 seeds	delivery $\geq$ 90 %	92.6 % (worst run 76 %); Table 16	Pass
TC-11	K3	kill the relay on the active path, 50 trials	re-route $\leq$ 10 s	mean 8.1 s; 23 of 50 within 10 s; worst 10.25 s; Figure 13	Partial
TC-12	K4, K6	congested radio; weighted versus hop count on identical seeds	SOS $\geq$ 3$\times$ faster; weighted delivers more	4.4$\times$; 99.2 % vs 57.4 %; Table 17	Pass

Table 21: Significant defects, causes, corrective actions and retesting

ID	Found by	Defect and cause	Fix and retest
D1	integration test of two components on one node	only one of discovery and relay received packets: one receive callback, the last registrant won	single dispatcher registered last; regression test
D2	Phase 4 integration	multi-hop ACK and retry failed: dedup keyed on message id dropped every ACK at the first relay and swallowed retries	key on type, id and attempt; retransmission counter in the reserved byte; re-ACK duplicates; new tests
D3	Phase 4 integration	dead nodes stayed routable: newer reports merged with old ones	a newer report replaces the origin's edges; tests
D4	fuzz test	a non-ASCII byte in a node-id field made `Packet.unpack` raise `UnicodeDecodeError` instead of `PacketError`; every receiver catches only `PacketError`, so one damaged packet could raise inside the receive path	decode and encode failures raise `PacketError`; both fuzz tests pass
D5	repeated runs of the socket tests	`Node.stop()` hung intermittently (in about a quarter of runs) on Windows with Python 3.10: `wait_for` swallowed a cancellation, so a background loop survived	helper built on `asyncio.wait`; 15 of 15 test-file runs and 40 of 40 standalone iterations clean
D6	repeated full-suite runs	timing-sensitive tests fail occasionally under load: `test_api::test_node_view_reports_peers_with_hop_distance` in 2 of 6 runs and `test_ack_multihop::test_ack_returns_over_the_path_the_data_took` in 1 of 6; both pass alone (8 of 8)	open: the fixed sleeps in these tests should become condition polling

9.9 Defect Identification and Resolution

D4 and D5 were found and fixed before the measurements in Chapter 8 were taken, so those results include the fixes.

9.10 Validation Against KPIs

Table 22: Validation of the key performance indicators of Table 2 against measured data

KPI	Target	Measured	Verdict
K1 Delivery	$\geq 90\,\%$ (40 nodes, 20 % loss, failures injected)	92.6 % mean over 50 runs (worst run 76 %). Without failures, a 43-node grid at 20 % loss: 95.8 %.	Met
K2 Convergence	$\leq 15\,\mathrm{s}$	collapse_40: mean 9.6 s; 47 of 50 runs $\leq 15\,\mathrm{s}$; worst 15.25 s. Mean 17.7 s in the 35 % loss scenario. Values come in multiples of the 5 s gossip interval. Live devices (2 s gossip): 3.8 s from process start.	Met in 47/50 runs; not at 35 % loss
K3 Recovery	$\leq 10\,\mathrm{s}$	mean 8.1 s, median 10.2 s; 23 of 50 trials $\leq 10\,\mathrm{s}$; the other 27 took 10.2–10.25 s, i.e. the 10 s heartbeat timeout plus the 0.25 s sampling step. Live devices (4 s timeout): median 4.0 s, max 4.5 s over 8 trials, every message delivered	Mean met; 27/50 trials miss by $\leq$ 0.25 s
K4 SOS advantage	$\geq 3\times$	4.4× (median routine / median SOS)	Met
K5 Store-and-forward	100 % flushed	201/201 in the no-gateway scenario; 100 % of live trials	Met
K6 Weighted vs hop count	Weighted better	99.2 % vs 57.4 %	Met
K7 Scale	≥ 50 nodes within the targets	50 nodes at 5 % loss: 100.0 % delivery; 50 nodes at 20 % loss: 88.2 %; largest tested 65 nodes: 98.9 % (5 %) and 86.0 % (20 %)	Met at 5 % loss; not at 20 %
K8 UI freshness	$\leq 2\,\mathrm{s}$	API response median 16 ms; screens poll every 0.7 s	Met
K9 Correctness	100 % passed	311/312 passed in all 3 runs; 1 timing-sensitive test failed once	Met except one flaky test
K10 Security	All hold	relay cannot decrypt, forged signature rejected, tampered payload rejected: automated tests pass	Met

K4, K5, K6 and K10 are met with margin; K1 is met in the specified condition; K2 and K3 are met on average but not in every run, by a fraction of a second, because of timer constants; K7 holds at 5 % loss but not at 20 % above about 43 nodes; K9 has one timing-sensitive test that failed once.

9.11 Limitations and Untested Scenarios

No radio hardware: the link is a neighbour-restricted LAN; interference, fading and mobility were not tested. **Live timing is loopback:** the five measured processes shared one machine, and the two-laptop run over a hotspot was a functional demonstration that was not timed. **No usability study:** the timed "screen-on to SOS-sent" test with five participants was not performed. **No automated UI tests. Scale:** nothing above 65 nodes and no long soak test. **Relays do not retain:** a relay with no route drops the packet (logged); only the originator retains, and its ACK timer covers the loss at the cost of a timeout. **Security:** one pre-shared key serves the mesh, so a node that holds it could read what it forwards (per-pair key agreement would remove this); no external review.

10 Results, Demonstration and Discussion

10.1 Implemented Features

Table 23: Feature status

Feature	Status	Evidence
Discovery, link-state gossip, weighted Dijkstra, priority, TTL and dedup, ACK and retry, store-and-forward, AES-GCM, Ed25519, SQLite log	Implemented	312 tests; Chapters 8, 9
Simulator, metrics, API, engine web application, survivor and rescue screens	Implemented	7 scenarios to 65 nodes; 12 API tests; Section 10.2
Multi-device deployment over real sockets	Implemented	5 processes measured; two-laptop demonstration
Immediate flush on a new neighbour; history filter	Partial	SNF-3, LOG-5
BLE or Wi-Fi Direct radio; per-pair key agreement; timed usability study	Not implemented	out of scope or future work

10.2 Screenshots and Demonstration Evidence

All screenshots are from the running system. Figures 14 and 15 come from the five-process live deployment in the demonstrated story: an SOS is delivered over a relay; the gateway fails and the next SOS is held; the gateway returns and the held message is delivered. The mesh was also run on two physical laptops joined by a phone hotspot (Figure 8): an SOS was delivered across both machines, the route changed when a relay was switched off, and a message held while the gateway was off was delivered when it returned. This run was a functional demonstration and was not timed.

10.3 Functional Results

Of the 68 requirements, 52 are verified by automated tests alone, 10 by automated tests plus a manual check of the screen, 2 by code inspection, 2 by manual check only, and 2 are partial (Table 5). The three stories that matter most were demonstrated on real devices: an SOS reaches the rescue device over several hops; the network re-routes around a failed relay; and a message with no route is held and later delivered.

10.4 Performance Results

Table 22 collects the measured KPIs: 92.6 % delivery on a 40-node mesh at 20 % loss with failures; an SOS median latency 4.4× lower under load; live SOS delivery in a median of 47 ms on loopback; relay failover in about 4 s; held messages flushed in about 4.5 s.

10.5 Comparison with Existing Systems

We did not run other systems, so the comparison is limited to documented properties (Table 4) and to the one baseline that can be measured fairly, hop-count routing on identical seeds (99.2 % against 57.4 % in the designed topology). We make no claim about the delivery of Bridgefy, Briar, Meshtastic or Serval, which publish no comparable measurements.

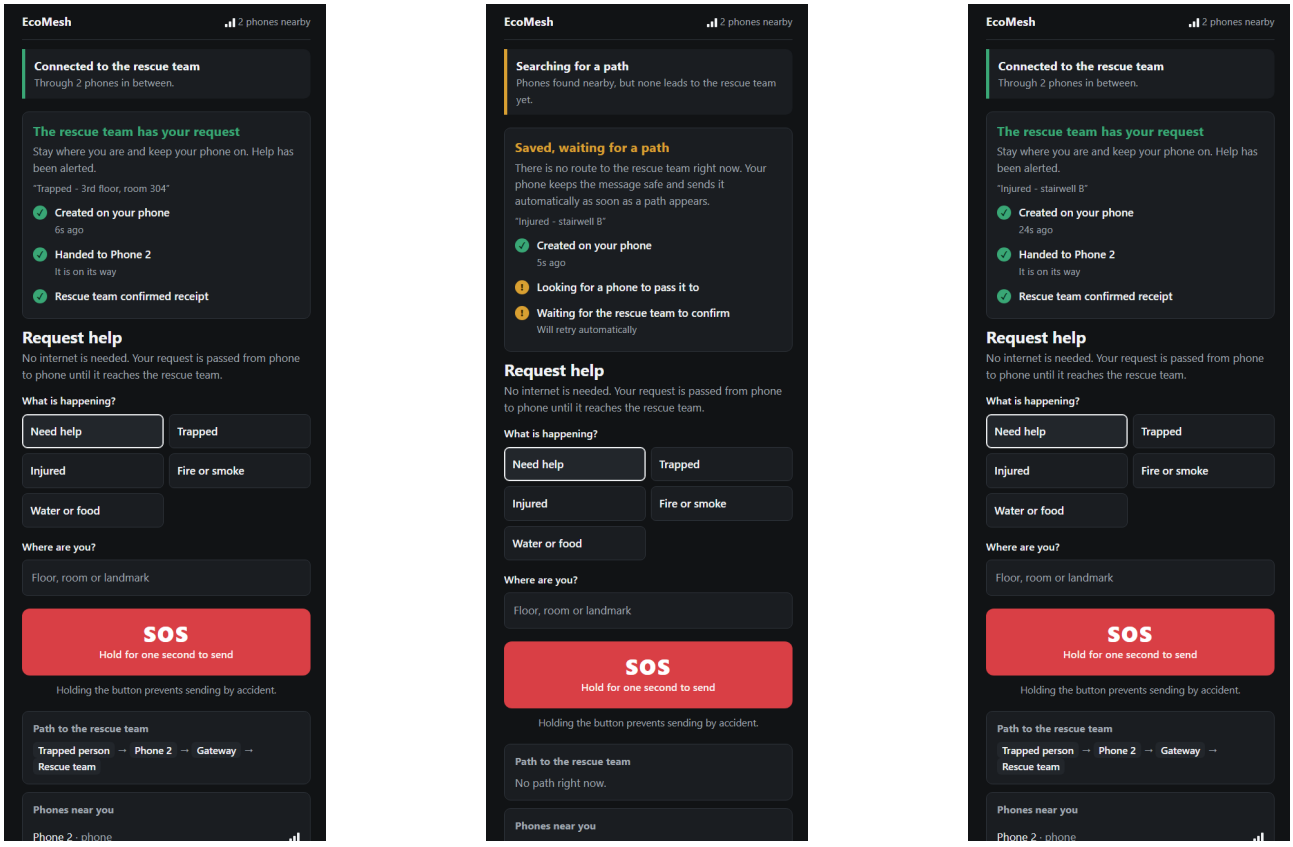


Figure 14: The survivor's screen. Left: SOS delivered over Phone 2 and the Gateway. Centre: the gateway has failed; the screen says "Searching for a path" and the message is "Saved, waiting for a path". Right: the gateway is back and the held message is delivered.



Figure 15: The rescue console. Left: SOS cards with sender, text, age and route, counters, network map and device list. Right: with the gateway down the rescue device hears nobody ("1 of 5 devices reachable"); earlier messages remain visible.

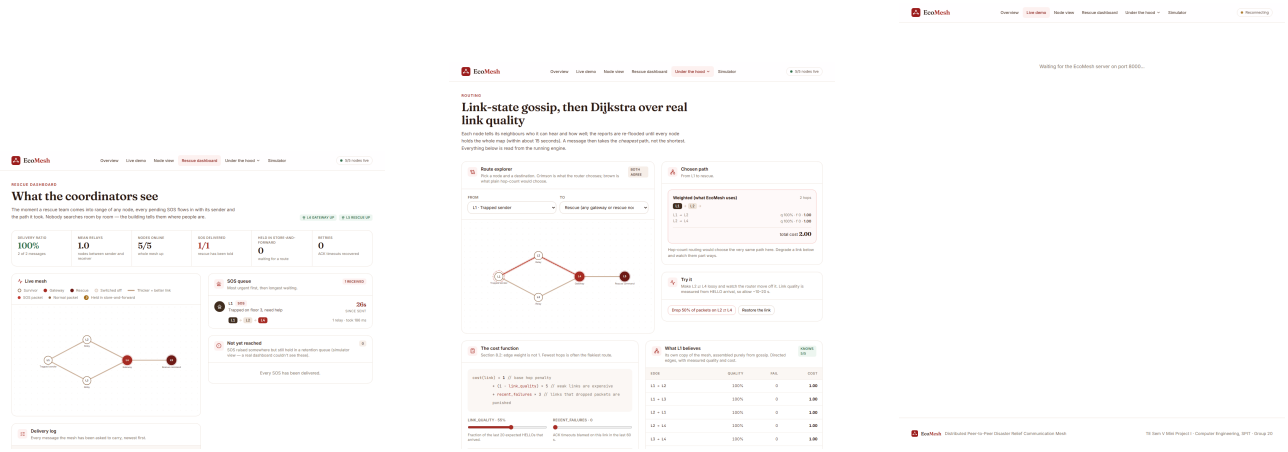


Figure 16: The engine web application. Left: rescue dashboard (delivery ratio, nodes online, live mesh, SOS queue). Centre: routing page (weighted versus hop-count route, cost function, a node's view of the graph). Right: the six-beat live demonstration page, which drives the real API.

10.6 Achievement of Objectives

Table 24: Objectives, supporting evidence and status

Obj.	Evidence	Status
O1	live convergence 3.8 s; simulator 5–15 s in multiples of the gossip interval	Achieved (47 of 50 runs $\leq$ 15 s)
O2	99.2 % vs 57.4 % on 20 identical seeds (Figure 13b)	Achieved
O3	4.4× lower median latency under load (Table 17)	Achieved
O4	92.6 % at 40 nodes and 20 % loss; recovery mean 8.1 s, 23 of 50 trials within 10 s	Partly achieved
O5	201 of 201 flushed in simulation; 4 of 4 live outages	Achieved
O6	security tests (Section 9.5)	Achieved with a pre-shared key
O7	every metric is a query over per-node logs	Achieved
O8	web application, device screens, API tests, five processes, two laptops	Achieved

10.7 Discussion of Findings

Hop count is a poor proxy for delivery, and the cost is large when it is wrong: a weak shortcut loses almost half the traffic under hop count and almost none under the weighted cost, with no change to topology or traffic. Timer constants shape the results: recovery clusters at the heartbeat timeout and convergence at multiples of the gossip interval, so those targets are properties of constants rather than algorithms, which tells us exactly what to tune. Loss is paid for in tail latency: delivery stays high to 30 % loss, but each lost ACK costs a timeout, so medians move from milliseconds to seconds. Priority matters only under contention: on an idle network SOS and routine traffic are equally fast, and the advantage appears once a link queues, which is the disaster condition. Testing found real defects in code we believed finished (Section 9.9).

10.8 Practical Applicability

A deployment needs devices running the node program, a topology (or multicast discovery), a gateway or rescue device and a shared network segment. It is usable today as a campus, hospital or building

safety system on existing laptops, and as the reference implementation of a protocol whose transport can later be Bluetooth Low Energy or Wi-Fi Direct. Without a radio transport it is not yet a phone-to-phone application, which is the main gap to deployment.

11 Conclusion and Future Scope

11.1 Conclusion

Every mainstream messaging tool fails when its infrastructure fails, although devices in a stricken building can still reach each other. We built EcoMesh, a peer-to-peer mesh in which every device is an endpoint and a relay, with a complete engine (discovery, link-state routing, reliability, security, storage), a simulator that measures it, a REST/WebSocket interface, two front ends, and a multi-device deployment of separate processes exchanging bytes over TCP. Weighted routing delivered 99.2 % of messages where hop-count routing delivered 57.4 % on identical seeds; an SOS had a 4.4× lower median latency under load; a 40-node mesh at 20 % loss with injected failures delivered 92.6 % over 50 runs; and on five live processes an SOS arrived in a median of 47 ms, the sender re-routed around a failed relay in about 4 s, and every message held during a gateway outage was delivered when the gateway returned.

11.2 Achievement of Objectives

Objectives O1–O3 and O5–O8 were achieved. O4 was achieved for delivery but only partly for recovery time, where 27 of 50 trials exceeded 10 s by at most 0.25 s because recovery is bounded by the 10 s heartbeat timeout; the evidence for each objective is in Table 24.

11.3 Major Engineering Contributions

A transport-agnostic engine that is identical on a simulator, on loopback and on separate laptops; a derived link-quality cost function with an immediate failure penalty and network-wide re-flooding; per-hop priority with a bounded queue that never evicts an SOS for routine traffic; signatures that survive multi-hop relaying; and a measurement framework in which every number in this report is generated from per-node logs and stored results.

11.4 Current Limitations

The link is a neighbour-restricted LAN rather than a radio; one pre-shared key serves the mesh; relays drop rather than retain when no route exists; recovery and convergence are quantised by timer constants; SNF-3 and LOG-5 are only partly met; there is no timed usability study or automated UI test; live timings were taken on loopback; and scale was tested to 65 simulated nodes (Section 9.11).

11.5 Future Enhancements

(1) A Bluetooth LE or Wi-Fi Direct transport and an Android client speaking the same protocol. (2) Shorter and adaptive timers: a neighbour timeout near 6–7 s, an immediate report on topology change and an ACK timeout derived from the measured round-trip time, re-measured with the same scripts. (3) X25519 per-pair key agreement. (4) Short retention at relays and an immediate flush on a new neighbour. (5) Per-destination route invalidation. (6) A usability study, automated browser tests and condition polling in timing-sensitive tests. (7) Tests beyond 65 nodes.

11.6 Lessons Learned

Build the simulator first and draw the transport boundary before the layers: it paid for itself in the simulator, the unit tests and the live deployment. Integration finds what unit tests do not (the multi-hop ACK bug survived every single-node test). Look at distributions as well as means: recovery clustered at the heartbeat timeout, which the mean hid. Record who built what from the first week, with consistent Git identities.

References

[1] M. R. Albrecht, J. Blasco, R. B. Jensen and L. Mareǩová, “Mesh Messaging in Large-scale Protests: Breaking Bridgefy,” in *Topics in Cryptology – CT-RSA 2021*, Lecture Notes in Computer Science, vol. 12704, Springer, 2021, pp. 375–398.

[2] C. E. Perkins and P. Bhagwat, “Highly dynamic Destination-Sequenced Distance-Vector routing (DSDV) for mobile computers,” in *Proc. ACM SIGCOMM*, 1994, pp. 234–244.

[3] D. Johnson, Y. Hu and D. Maltz, “The Dynamic Source Routing Protocol (DSR) for Mobile Ad Hoc Networks for IPv4,” RFC 4728, IETF, Feb. 2007.

[4] C. Perkins, E. Belding-Royer and S. Das, “Ad hoc On-Demand Distance Vector (AODV) Routing,” RFC 3561, IETF, July 2003.

[5] T. Clausen and P. Jacquet (eds.), “Optimized Link State Routing Protocol (OLSR),” RFC 3626, IETF, Oct. 2003.

[6] J. Moy, “OSPF Version 2,” RFC 2328, IETF, Apr. 1998.

[7] J. Chroboczek and D. Schinazi, “The Babel Routing Protocol,” RFC 8966, IETF, Jan. 2021.

[8] D. S. J. De Couto, D. Aguayo, J. Bicket and R. Morris, “A high-throughput path metric for multi-hop wireless routing,” in *Proc. ACM MobiCom*, 2003, pp. 134–146.

[9] A. Vahdat and D. Becker, “Epidemic Routing for Partially-Connected Ad Hoc Networks,” Technical Report CS-2000-06, Duke University, 2000.

[10] T. Spyropoulos, K. Psounis and C. S. Raghavendra, “Spray and Wait: An efficient routing scheme for intermittently connected mobile networks,” in *Proc. ACM SIGCOMM Workshop on Delay-Tolerant Networking (WDTN)*, 2005, pp. 252–259.

[11] A. Lindgren, A. Doria and O. Schelén, “Probabilistic routing in intermittently connected networks,” *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 7, no. 3, pp. 19–20, 2003.

[12] K. Fall, “A delay-tolerant network architecture for challenged internets,” in *Proc. ACM SIGCOMM*, 2003, pp. 27–34.

[13] V. Cerf *et al.*, “Delay-Tolerant Networking Architecture,” RFC 4838, IETF, Apr. 2007.

[14] L. Pelusi, A. Passarella and M. Conti, “Opportunistic networking: data forwarding in disconnected mobile ad hoc networks,” *IEEE Communications Magazine*, vol. 44, no. 11, pp. 134–141, 2006.

[15] D. G. Reina, M. Askalani, S. L. Toral, F. Barrero, E. Asimakopoulou and N. Bessis, “A survey on multihop ad hoc networks for disaster response scenarios,” *International Journal of Distributed Sensor Networks*, vol. 11, no. 10, art. 647037, 2015.

[16] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, Nov. 2007.

[17] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[18] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[19] The Briar Project, “Briar: secure messaging anywhere – how it works,” `https://briarproject.org`, accessed 2026.

[20] Meshtastic, “Mesh Broadcast Algorithm” and “Why Meshtastic uses managed flood routing,” documentation, `https://meshtastic.org/docs/overview/mesh-algo/`, accessed 2026.

[21] The Serval Project, “Serval: communications for disaster and remote areas” (including the Rhizome data-distribution protocol), project documentation, `https://www.servalproject.org`, accessed 2026.